CAMPUS FACILITY AND MAINTENANCE MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

JANNANI R

(2116241401031)

in partial fulfilment of the End Semester Examination for the course

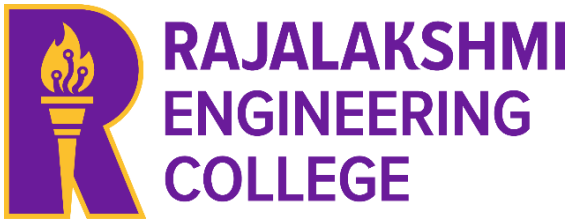
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

[BACHELOR OF TECHNOLOGY]

Computer Science and Business Systems



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

[2026–2027]

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**Campus Facility and Maintenance Management System**" is the bonafide work of **JANNANI R** (2116241401031), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026-2027, Semester V, under my supervision.

SIGNATURE

Dr. K.R. Sowmia, PhD
HEAD OF THE DEPARTMENT
Department of Computer Science and Business Systems, Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Alphin Ezhil Manuel, ML.
COURSE FACULTY
Assistant Professor(SG)
Department of Computer Science and Business System
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on [ ].

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, JANNANI R (241401031), hereby declare that the project report titled "**Campus Facility and Maintenance Management System**", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
JANNANI R
2116241401031

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, **Mr. S. Meganathan, B.E., F.I.E.,** Chairperson **Dr. Thangam Meganathan, Ph.D.,** and Vice-Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.,** for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, **Dr. S.N. Murugesan, M.E., Ph.D,** Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank **Dr. K.R. Sowmia, Ph.D.,** Head of the Department, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, **Alphin Ezhil Manuel, ML.,** for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation / Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

JANNANI R

ABSTRACT

Campus facility maintenance involves handling complaints related to classrooms, laboratories, libraries, and other facilities, and managing them manually makes it difficult to track requests, assign technicians, monitor repairs, and maintain records. The Campus Facility and Maintenance Management System was developed using Zoho Creator to organize these activities through a centralized application with six forms (Submit New Complaint, Facility Register, Technician Register, Work Assignment, Repair Update, and Feedback), reports, and a Campus Maintenance Overview dashboard with charts and a Board grouped by issue category. Deluge scripting handles complaint submission confirmation and work assignment date validation, a repair validation workflow requires a completion date before a repair is marked as resolved, and email notifications and role-based access are provided for Administrator, Technician, and Requester. Testing with sample records, form submissions, validation checks, and access permissions confirmed the date validation, but technicians could not view certain assignments created by administrators, so further refinement is needed. Future improvements include better assignment visibility, overdue alerts, and preventive maintenance scheduling; quantitative improvements in repair turnaround time have not yet been measured.

Keywords: Zoho Creator; Low-Code Development; Deluge Scripting; Workflow Automation; Campus Maintenance.

PROJECT AT A GLANCE

Attribute	Details
Project Title	Campus Facility and Maintenance Management System
Application Domain	Education
Student Name & Register No.	R Jannani – [2116241401031]
Zoho Creator Edition	Education
Application Name (Link Name)	Campus Facility and Maintenance Management System
Published / Live URL	Live URL
Demo Credentials (per role)	Administrator | Role 1: Operator | Role 2: Student/Staff
Number of Forms / Subforms	6 Forms / 1 subform
Number of Reports (by type)	List: All Complaints and other configured list reports; Board/Kanban: Complaints grouped by Issue Category; Others: Dashboard
Number of Pages / Dashboards	1 confirmed dashboard – Campus Maintenance Overview.
Number of Workflows	Form: 3 confirmed workflows; Schedule: 0 confirmed; Approval: 0 confirmed; Blueprint: 0 confirmed; Batch: 0 confirmed
Deluge Functions / Custom Actions	3 confirmed scripts – DS-01 Work Assignment Date Validation, DS-02 Repair Completion Validation, DS-03 Complaint Submission Confirmation
Integrations	Zoho Sheet – complaint data synchronization using `zoho.sheet.createRecords`
Roles & Profiles Defined	Administrator and Operator are intended roles; student/staff access through the published complaint form. Verify configured profiles and permissions.
Environments Used	Development → Stage → Production
Date of Production Deployment	19 October 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 6
REQUIREMENT ANALYSIS 7
3.1 Stakeholders and User Roles 7
3.2 Functional Requirements 7
3.3 Non-Functional Requirements 8
3.4 Hardware and Software Requirements 8
3.5 Use Case Model 9
SYSTEM DESIGN 10
4.1 System Architecture 10
4.2 Data Model 10
4.3 Form Design 12
4.4 Process and Workflow Design 16
4.5 User Interface and Navigation Design 18
4.6 Access Control Design 21
IMPLEMENTATION 10
5.1 Application Setup 10
5.2 Forms and Data Capture 10

5.3 Reports 11
5.4 Pages and Dashboards 12
5.5 Deluge Scripting 13
5.6 Workflows and Automation 14
5.7 Integrations 15
5.8 User Management and Role-Based Access 16
5.9 AI and Analytics Features 17
5.10 Pro-Code Extension 18
5.11 Mobile and Portal Access 18
TESTING AND DEBUGGING 19
6.1 Testing Strategy 19
6.2 Test Cases 19
6.3 Debugging Deluge Scripts 20
6.4 Defect Log 21
6.5 User Acceptance Testing 21
DEPLOYMENT, SECURITY AND GOVERNANCE 22
7.1 Environments and Deployment Strategy 22
7.2 Performance Optimisation 23
7.3 Security Features 24
7.4 Governance and Administration 24
RESULTS AND DISCUSSION 26
8.1 Application Output 26
8.2 Achievement of Objectives 28
8.3 Comparison with the Existing System 29
8.4 Limitations of the Low-Code Approach 29
8.5 Reflection on the No-Code · Low-Code · Pro-Code Spectrum 30
CONCLUSION AND FUTURE ENHANCEMENTS 31
9.1 Conclusion 31
9.2 Future Enhancements 31
9.3 Learning Outcomes and Career Relevance 31
REFERENCES 32
APPENDIX A · ZOHO CREATOR FEATURE COVERAGE CHECKLIST 33
APPENDIX B · COURSE OUTCOME MAPPING 36
APPENDIX C · COMPLETE DELUGE SOURCE CODE 37
APPENDIX D · USER MANUAL 38
D.1 Accessing the Application 38
D.2 Instructions for Administrator 38

D.3 Instructions for Student/Staff 38

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 4
Table 2.2: Summary of Literature Survey 5
Table 3.1: Stakeholders and User Roles 7
Table 3.2: Functional Requirements 7
Table 3.3: Non-Functional Requirements 8
Table 3.4: Hardware and Software Requirements 8
Table 3.5: Use Case Description · [Use Case Name.] 9
Table 4.1: Entities and Relationships 11
Table 4.2: Field Specification · Submit New Complaint Form 12
Table 4.3: Field Specification · Facility Register Form 13
Table 4.4: Field Specification · Technician Register Form 13
Table 4.5: Field Specification · Work Assignment Form 14
Table 4.6: Field Specification · Repair Update Form 14
Table 4.7: Field Specification · Feedback Form 15
Table 4.8: Role·Permission Matrix 21
Table 5.1: Reports Implemented 11
Table 5.2: Deluge Scripts Implemented 13
Table 5.3: Workflows Implemented 14
Table 5.4: Integrations Implemented 15
Table 6.1: Test Cases 19
Table 6.2: Defect Log 21
Table 6.3: User Acceptance Feedback 21
Table 7.1: Deployment Checklist 22
Table 7.2: Performance Optimisation Results 23
Table 7.3: Security Controls Implemented 24
Table 8.1: Mapping of Objectives to Outcomes 28
Table 8.2: Existing System versus Proposed Application 29
Table A.1: Feature Coverage Checklist 33
Table B.1: Mapping of Course Outcomes to Report Chapters 36

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System 3
Figure 3.1: Use Case Diagram of the Application 9
Figure 4.1: System Architecture of the Application 10
Figure 4.2: Entity·Relationship Diagram 10
Figure 4.3: Complaint Submission Workflow 16
Figure 4.4: Work Assignment Validation Workflow 16
Figure 4.5: Repair Completion Validation Workflow 17
Figure 4.6: Operator Assignment Notification Workflow 17
Figure 4.7: Navigation Structure of the Application 18
Figure 4.8: Campus Maintenance Overview Dashboard 18
Figure 4.9: Submit New Complaint Form 19
Figure 4.10: Work Assignment Form 20
Figure 4.11: Repair Update Form 20
Figure 5.1: Application Home and Navigation Sections 10
Figure 5.2: [Submit New Complaint] Form in the Form Builder 11
Figure 5.3: Submit New Complaint Form in Live Mode 11
Figure 5.4: All complaint Report 12
Figure 5.5: Maintanance Dashboard 12
Figure 5.6: Validate_work_Assignment_Dates Configuration 15
Figure 5.7: Zoho_sheet_connection 16
Figure 5.8: User profiles(RBAC) & roles 17
Figure 5.9: AI / Analytics Feature in Use 18
Figure 6.1: Deluge Debug 21
Figure 7.1: Environment 22
Figure 7.2: Version History and Application Administration in Zoho Creator 25
Figure 8.1: Scenario Step 1 · Complaint Submission and Confirmation 26
Figure 8.2: Scenario Step 2 ·Complaint Review and Work Assignment 27
Figure 8.3: Scenario Step 3 · Repair Update and Dashboard Monitoring 28

LIST OF ABBREVIATIONS.

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
JSON	JavaScript Object Notation
LCAP	Low-Code Application Platform
OAuth	Open Authorization
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
SSO	Single Sign-On
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Educational institutions depend on facilities such as classrooms, computer laboratories, libraries, and restrooms. These facilities require regular maintenance to ensure their proper functioning. Maintenance complaints must be recorded, assigned to technicians, and monitored until the problems are resolved.

The Campus Facility and Maintenance Management System is developed using Zoho Creator to organize these activities. It includes forms for complaint submission, facility registration, technician registration, work assignment, repair updates, and feedback. Reports, dashboards, and Deluge workflows help manage and monitor maintenance information.

1.2 Problem Statement

Managing maintenance complaints through manual records and separate spreadsheets can make it difficult to track requests, assign technicians, and monitor repair progress. A centralized application is required to organize maintenance information and improve record management.

The major problems are:

- Maintenance records may be scattered across different sources.
- Tracking complaint status requires manual effort.
- Assignment and repair information may be difficult to access.
- Incorrect dates and incomplete repair details can cause errors.
- User access and record visibility require proper configuration.

1.3 Objectives

- To develop six forms for managing complaints, facilities, technicians, assignments, repairs, and feedback.
- To connect related records using lookup fields.
- To automate complaint confirmation and validate assignment dates using Deluge.
- To configure repair validation and email notifications.
- To create reports and a dashboard for monitoring maintenance information.

1.4 Scope of the Project

- The project covers complaint management, facility and technician registration, work assignments, repair updates, feedback, reports, dashboards, and workflow automation using Zoho Creator.
- The application includes Administrator, Technician, and Requester roles. Testing covers sample records, validation workflows, and access permissions.
- External API integrations, preventive maintenance scheduling, advanced analytics, and production deployment are outside the current scope.

1.5 Justification for a Low-Code Approach

Zoho Creator is suitable for this project because it provides visual tools to create forms, reports, dashboards, workflows, and access permissions with less conventional programming. Deluge scripting supports custom validation and automation. These features simplify application development and maintenance while allowing future improvements.

The platform also provides hosting and access management capabilities, although permissions must be configured and tested carefully.

Reference: Zoho Corporation, "Zoho Creator," https://www.zoho.com/creator/.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing campus maintenance process begins when a student, faculty member, or staff member reports a facility issue. The complaint is recorded manually, and the responsible person assigns a technician. The technician performs the maintenance work and checks whether the issue has been resolved. If the repair is incomplete, further maintenance is required. Once completed, the status is updated, feedback may be collected, and reports are prepared manually.

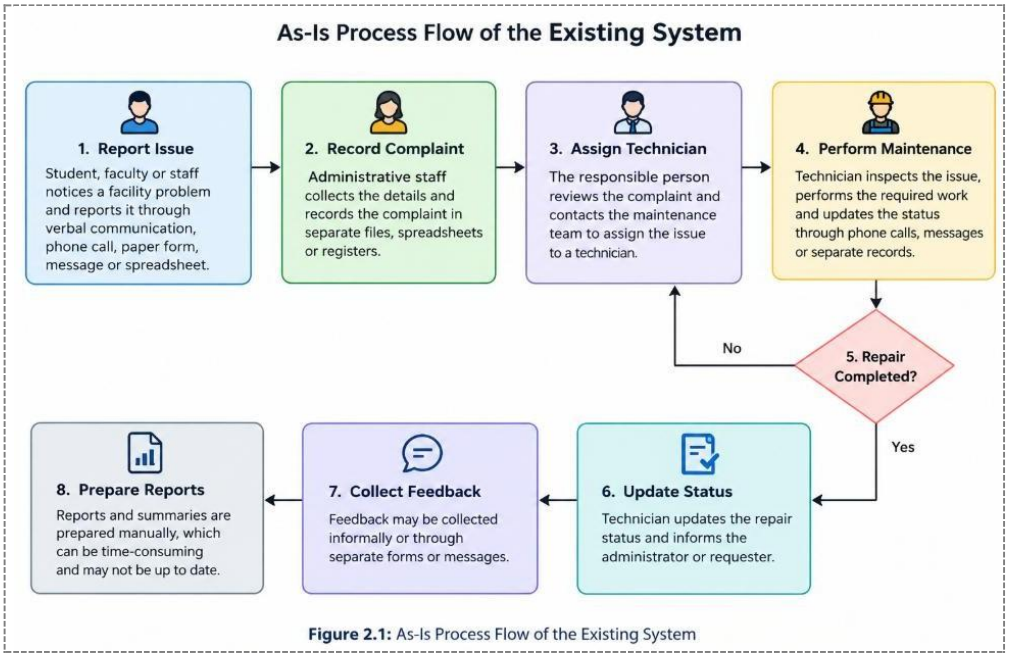


Figure 2.1: As-Is Process Flow of the Existing System

2.2 Limitations of the Existing System

Scattered Records: Maintenance complaints stored in paper records, messages, or separate spreadsheets are difficult to organize and retrieve.

Manual Tracking: Assigning technicians and monitoring repair progress require manual coordination and may cause delays.

Limited Visibility: Preparing reports and checking complaint status can be time-consuming, making it difficult to monitor maintenance activities efficiently.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Estimated 6–10 weeks for forms, database, workflows, reports, and access control.	The application can be developed faster using Zoho Creator's drag-and-drop form builder and built-in components.
Skills required	Programming, database design, frontend and backend development.	Visual form design, workflow configuration, and basic Deluge scripting.
Hosting & infrastructure	Requires application hosting and infrastructure configuration.	Hosting facilities are provided by the platform.
Mobile application	Requires responsive development or separate mobile development.	Supports mobile access through Zoho Creator's platform capabilities.
Security & access control	Must be implemented and configured through application code and infrastructure.	Provides built-in permission and sharing settings that require proper configuration.
Maintenance & change	Changes may require code modifications, testing, and redeployment.	Forms, reports, and workflows can be modified through visual tools.
Customisation limits	Offers greater flexibility for complex, specialised requirements.	Customisation depends on platform features and Deluge capabilities.
Cost	May involve development, hosting, infrastructure, and maintenance expenses.	Depends on the Zoho Creator plan, user requirements, and usage.

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Bucaioni et al., 2022	Modelling in Low-Code Development: A Multi-Vocal Systematic Review	Reviews low-code development research and its relationship with model-driven engineering.	Supports the use of visual application design to build maintenance forms and workflows.
2	Pinho et al., 2023	What About the Usability in Low-Code Platforms? A Systematic Literature Review	Examines usability in low-code platforms and highlights the importance of user experience.	Supports the use of structured forms and reports for easier maintenance record management.
3	Gartner, 2024	*Magic Quadrant for Enterprise Low-Code Application Platforms*	Reviews enterprise low-code platforms and their capabilities for application development and maintenance.	Provides industry context for selecting a low-code development platform.
4	Zoho Corporation, n.d.	*Zoho Creator Documentation: Form Workflows*	Explains how workflows automate actions triggered by form submissions and record events.	Supports complaint confirmation, date validation, and email notification workflows.
5	Zoho Corporation, n.d.	*Zoho Creator Documentation: Understanding Permissions*	Describes permission settings for controlling access to application components and records.	Supports the Administrator, Technician, and Requester access configuration.

2.5 Emerging Trends in Low-Code Development

Low-code development is evolving through trends such as AI-assisted app building, citizen development, hyper-automation, and composable applications.

- AI-Assisted App Building: AI tools can help users generate forms, suggest workflows, and simplify application development.
- Citizen Development: Low-code platforms allow users with limited programming knowledge to create applications using visual tools. Zoho Creator helped simplify the development of the campus maintenance system.
- Hyper-Automation: Automating repetitive tasks reduces manual effort. In this project, Deluge scripts and email workflows support complaint confirmation, date validation, and notifications.
- Composable Applications: Applications can be developed by combining reusable components such as forms, reports, dashboards, and workflows. The project uses these components to manage complaints, technician assignments, and repair updates.

These trends influenced the design by encouraging a modular, user-friendly application with automated processes and centralized maintenance records.

2.6 Proposed System

The proposed system is a Campus Facility and Maintenance Management System developed using Zoho Creator. It provides a centralized platform to submit complaints, maintain facility and technician records, assign maintenance work, update repair status, and collect feedback.

The system overcomes the limitations of the existing manual process in the following ways:

Scattered Records: Stores complaint and maintenance details in a centralized application.

- **Manual Tracking:** Uses reports, dashboards, workflows, and email notifications to support complaint tracking and communication.

- **Limited Visibility:** Provides role-based access for administrators, technicians, and requesters to access permitted information.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Manages the application.	Manage users, configure settings, and view all data.
Operator	Handles maintenance work.	View permitted information and update repair details.
User(Requester)	Reports campus maintenance issues.	Submit complaints and provide feedback.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow users to submit campus maintenance complaints.	Requester	High
FR-02	The system shall validate work assignment dates before submission.	Administrator	High
FR-03	The system shall send email notifications for relevant maintenance activities.	Administrator, Operator	Medium
FR-04	The system shall allow administrators to assign maintenance work to technicians.	Administrator	High
FR-05	The system shall display maintenance details through dashboards and reports.	Administrator	Medium
FR-06	The system shall allow Operator to update repair status and completion details.	Operator	High
FR-07	The system shall allow requesters to submit feedback on maintenance work.	Requester	Medium
FR-08	The system shall validate the completion date before marking a repair as resolved.	Operator	High
FR-09	The system shall integrate with external services or APIs to support data exchange.	Administrator	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports should load efficiently for normal application usage.
NFR-02	Security	Users shall access only the data permitted by their roles.
NFR-03	Usability	The application should be easy to use through a web browser.
NFR-04	Availability	The application should be accessible when the Zoho Creator service and internet connection are available.
NFR-05	Maintainability	Forms, reports, and workflows should be easy to update and maintain.
NFR-06	Auditability	Record changes should be traceable wherever audit history is supported.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Postman
Other Zoho services	Zoho Creator email workflows
Mobile	Zoho Creator Desktop
Client hardware	Intel Core i3/i5 Laptop or desktop and internet connection

3.5 Use Case Model

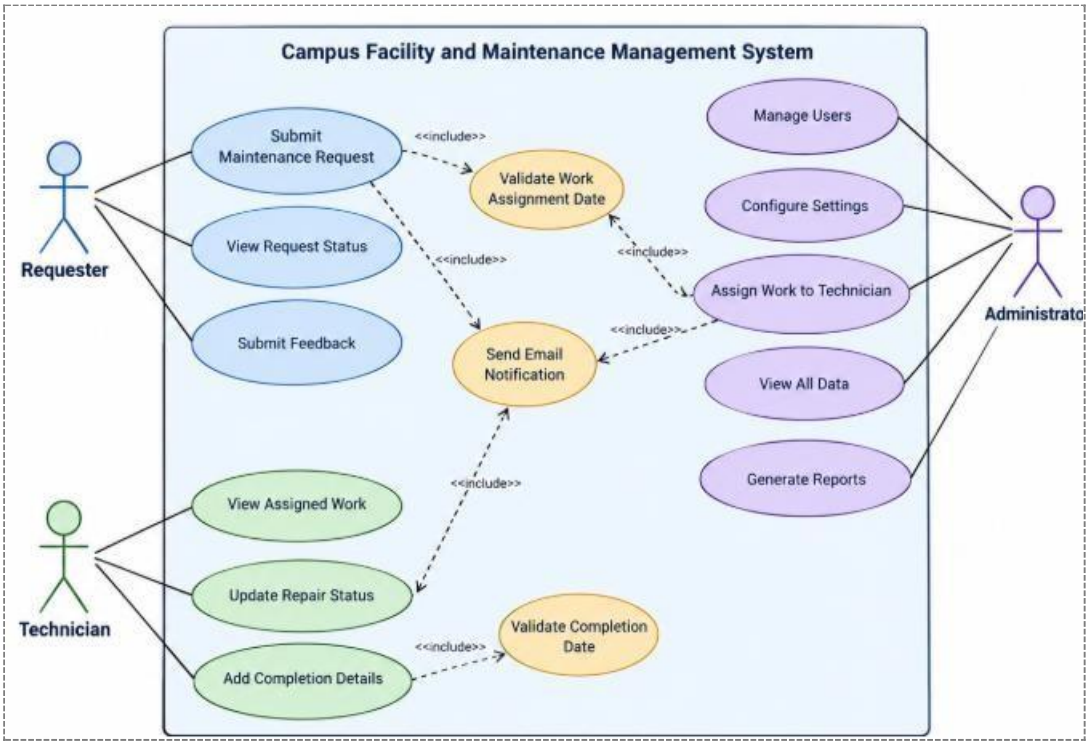


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – [Use Case Name]

Use Case ID	UC-01
Use Case Name	Submit Complaint
Actor(s)	Requester
Pre-condition	The user has access to the application.
Main Flow	1. User opens the complaint form. 2. Enters complaint details. 3. Submits the form. 4. System saves the complaint and displays a confirmation message.
Alternate Flow	If mandatory details are missing, the user must correct them before submission.
Post-condition	The complaint is saved in the system.
Related FR	FR-01

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

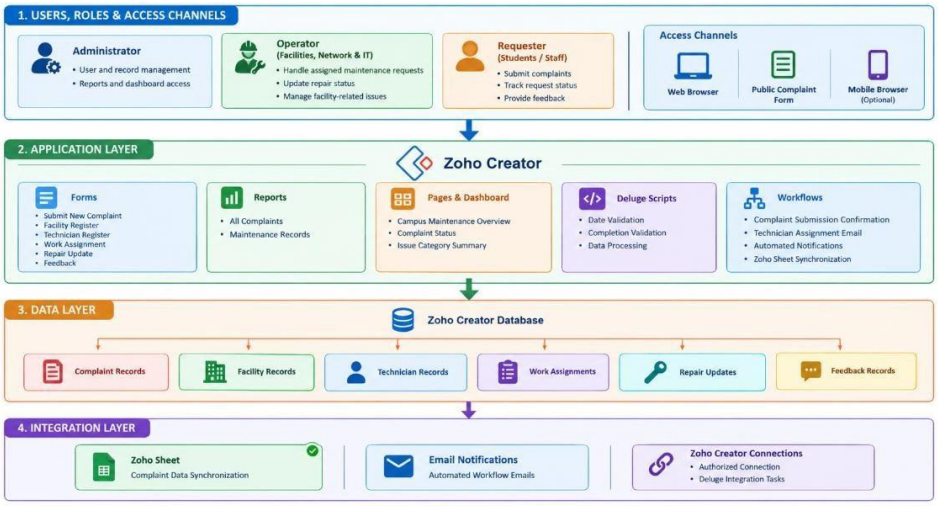


Figure 4.1: System Architecture of the Appification

Figure 4.1: System Architecture of the Application

The system architecture consists of four layers: the user layer, application layer, data layer, and integration layer. Zoho Creator manages forms, reports, dashboards, and workflows, while the data layer stores maintenance records. Zoho Sheet integration and automated email notifications support data synchronization and communication between users.

4.2 Data Model

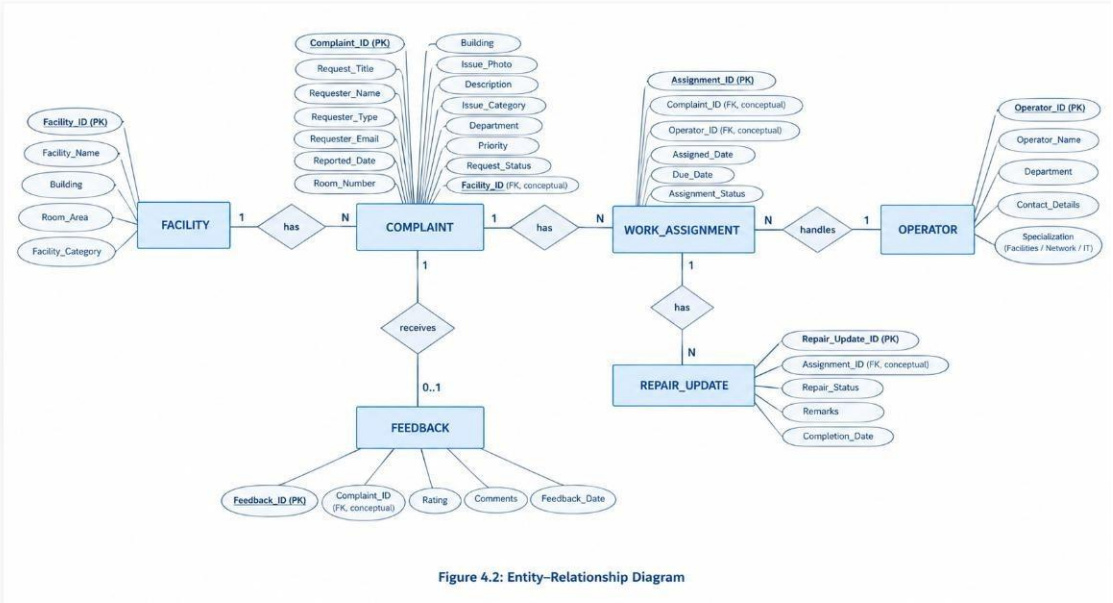


Figure 4.2: Entity–Relationship Diagram

Figure 4.2: Entity–Relationship Diagram

The data model defines the main entities of the Campus Facility and Maintenance Management System and the relationships between them. It includes Complaint, Facility, Operator, Work Assignment, Repair Update, and Feedback. In Zoho Creator, these relationships are managed using lookup fields and subforms, helping organize maintenance records and track complaints from submission to resolution.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Complaint	Complaint_ID, Request_Title, Requester_Name, Requester_Email, Request_Status	Facility	One-to-Many	Lookup field
Facility	Facility_ID, Facility_Name, Building, Room_Area	Complaint	One-to-Many	Lookup field in Complaint
Operator	Operator Name, Department, Specialization, Contact Details	Work Assignment	One-to-Many	Lookup in Work Assignment
Work Assignment	Assigned Date, Due Date, Assignment Status	Complaint, Operator	Many-to-One	Lookup fields & subform
Repair Update	Repair Status, Remarks, Completion Date	Work Assignment	Many-to-One	Lookup or subform, as implemented
Feedback	Rating, Comments, Feedback Date	Complaint	One-to-Many	Lookup, if configured

4.3 Form Design

This section lists the field specification of each form in the application. Field types, link names, and validation rules are given as recorded for the application; details that were not available are marked "Not provided".

Table 4.2: Field Specification – Submit New Complaint Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Request Title	Request_Title	Single Line	Not provided	Not provided	Complaint title
Requester Name	Requester_Name	Single Line	Not provided	Not provided	Requester details
Requester Type	Requester_Type	Drop Down	Not provided	Not provided	Requester role
Requester Email	Requester_Email	Email	Not provided	Valid email format	Contact email
Reported Date	Reported_Date	Date	Not provided	Not provided	Complaint date
Room Number	Room_Number	Single Line	Not provided	Not provided	Room location
Building	Building	Not provided	Not provided	Not provided	Building location
Issue Photo	Image_If_applicable	Not provided	Not provided	Not provided	Optional issue photo
Description	Description	Not provided	Not provided	Not provided	Details of the issue
Issue Category	Issue_Category	Not provided	Not provided	Not provided	Complaint category
Department	Department	Not provided	Not provided	Not provided	Relevant department
Priority	Priority	Not provided	Not provided	Not provided	Complaint urgency
Request Status	Request_status	Not provided	Not provided	Not provided	Current complaint status

Table 4.3: Field Specification – Facility Register Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Facility Name	Not provided	Single Line	Not provided	Not provided	Name of the facility
Building	Not provided	Not provided	Not provided	Not provided	Building where facility is located
Room / Area	Not provided	Single Line	Not provided	Not provided	Facility location
Facility Category	Not provided	Drop Down	Not provided	Not provided	Category of facility
Description	Not provided	Multi Line	Not provided	Not provided	Additional facility details

Table 4.4: Field Specification – Technician Register Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Operator Name	Not provided	Not provided	Not provided	Not provided	Name of the maintenance operator
Department	Not provided	Drop Down	Not provided	Not provided	Facilities, Network, or IT
Contact Details	Not provided	Phone	Not provided	Not provided	Operator contact information
Email	Not provided	Email	Not provided	Valid email format	Operator email address
Specialization	Not provided	Drop Down	Not provided	Not provided	Area of maintenance expertise

Table 4.5: Field Specification – Work Assignment Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Complaint	Not provided	Lookup	Not provided	Select the relevant complaint	Complaint assigned for maintenance
Technician / Operator	Not provided	Lookup	Not provided	Select the relevant operator	Assigned maintenance operator
Assigned Date	Assigned_Date	Date	Not provided	Must be valid; compared with Due Date	Date work is assigned
Due Date	Due_Date	Date	Not provided	Cannot be earlier than Assigned Date	Target completion date
Assignment Status	Not provided	Drop Down	Not provided	Not provided	Current assignment status
Maintenance Tasks	Not provided	Subform	Not provided	Not provided	Subform for maintenance task details

Table 4.6: Field Specification – Repair Update Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Work Assignment	Not provided	Lookup	Not provided	Not provided	Related maintenance assignment
Repair Status	Repair_Status	Drop Down	Not provided	Not provided	Current repair status
Remarks	Not provided	Multi Line	Not provided	Not provided	Details of repair work
Completion Date	Completion_Date	Date	Conditional	Required when Repair Status is Resolved	Date repair was completed

Table 4.7: Field Specification – Feedback Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Complaint	Not provided	Lookup	Not provided	Not provided	Complaint related to feedback
Rating	Not provided	Not provided	Not provided	Not provided	User rating for maintenance service
Comments	Not provided	Multi Line	Not provided	Not provided	User feedback and suggestions
Feedback Date	Not provided	Date	Not provided	Not provided	Date feedback was submitted

4.4 Process and Workflow Design

The automated processes of the application are modelled as flowcharts. The darker blue boxes mark the steps where Deluge scripts or Zoho Creator workflows execute.

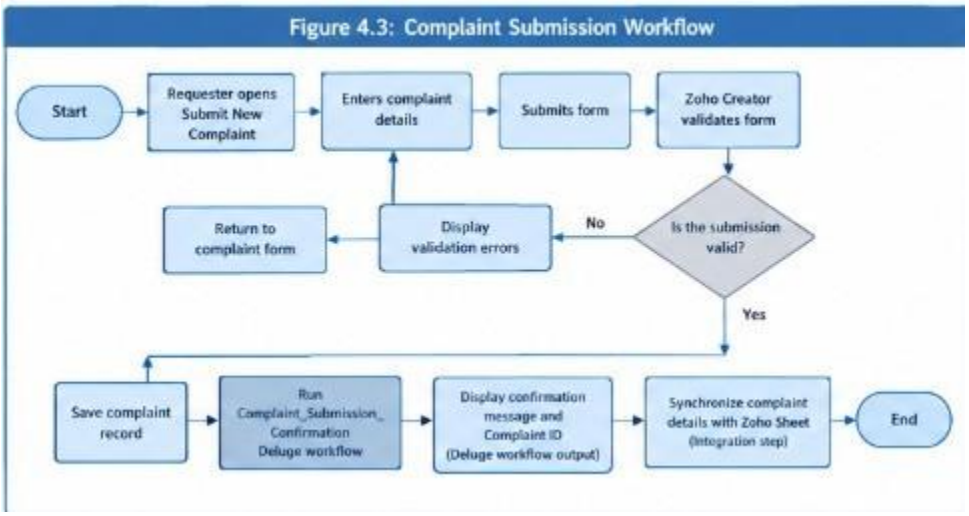


Figure 4.3: Complaint Submission Workflow

The requester submits the Submit New Complaint form and Zoho Creator validates it. If it is valid, the complaint record is saved and the Complaint_Submission_Confirmation Deluge workflow displays a confirmation message with the Complaint ID. The complaint details are then synchronized with Zoho Sheet through the integration step.

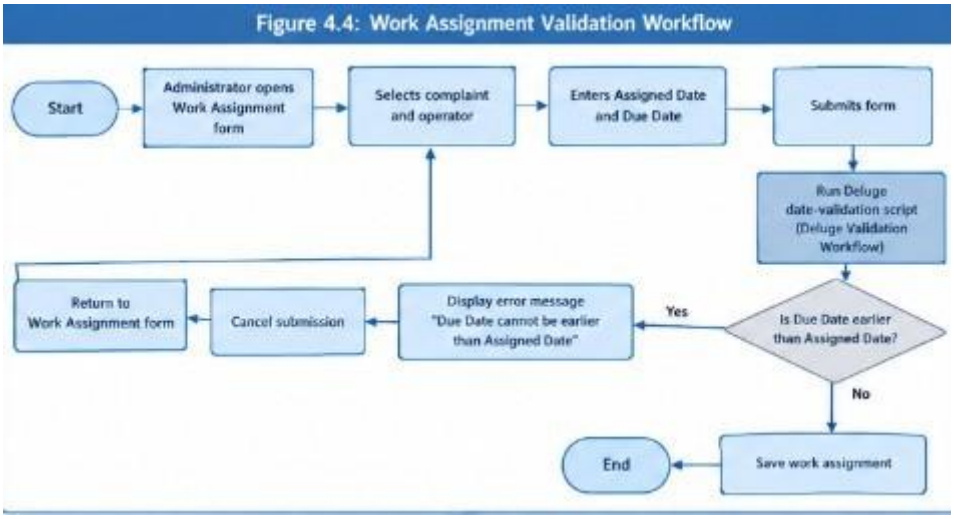


Figure 4.4: Work Assignment Validation Workflow

When the administrator submits a work assignment, a Deluge date-validation script compares the Due Date with the Assigned Date. If the Due Date is earlier, an error message is displayed and the submission is cancelled; otherwise the work assignment is saved.

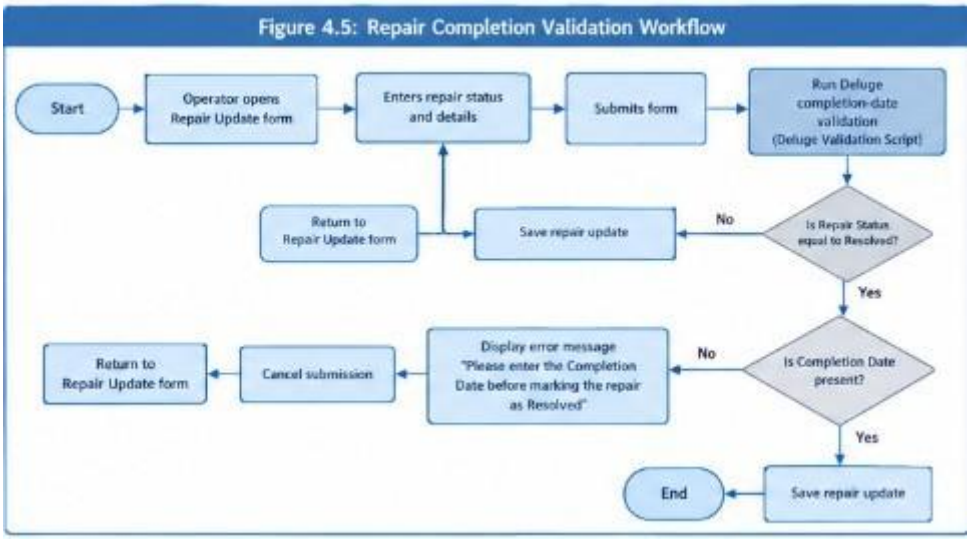


Figure 4.5: Repair Completion Validation Workflow

When the operator submits a repair update with the status Resolved, a Deluge validation script checks that the Completion Date is present. If it is missing, an error message is displayed and the submission is cancelled; otherwise the repair update is saved.

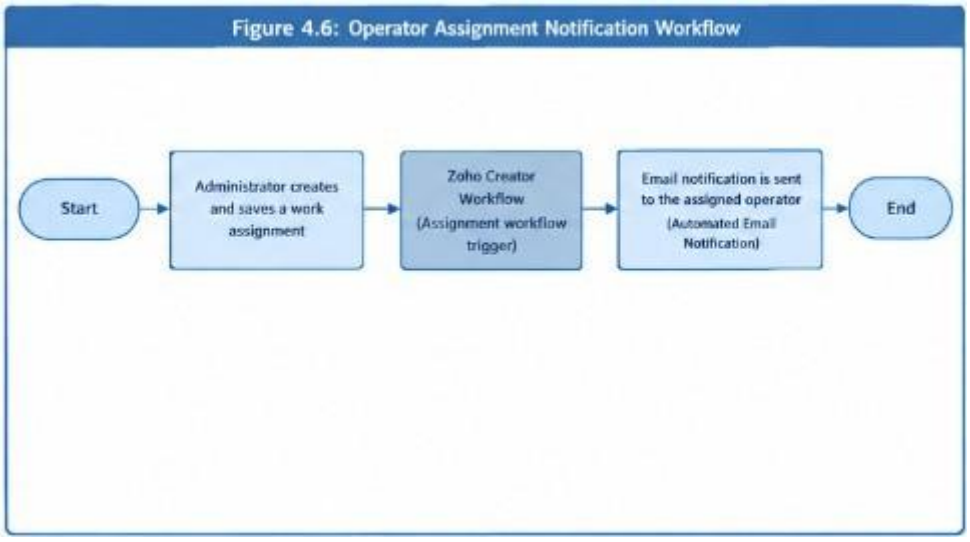


Figure 4.6: Operator Assignment Notification Workflow

After the administrator saves a work assignment, a Zoho Creator workflow is triggered and sends an automated email notification to the assigned operator.

4.5 User Interface and Navigation Design

This section presents the navigation structure of the application, and the implemented dashboard, complaint form, work assignment form, and repair update form.

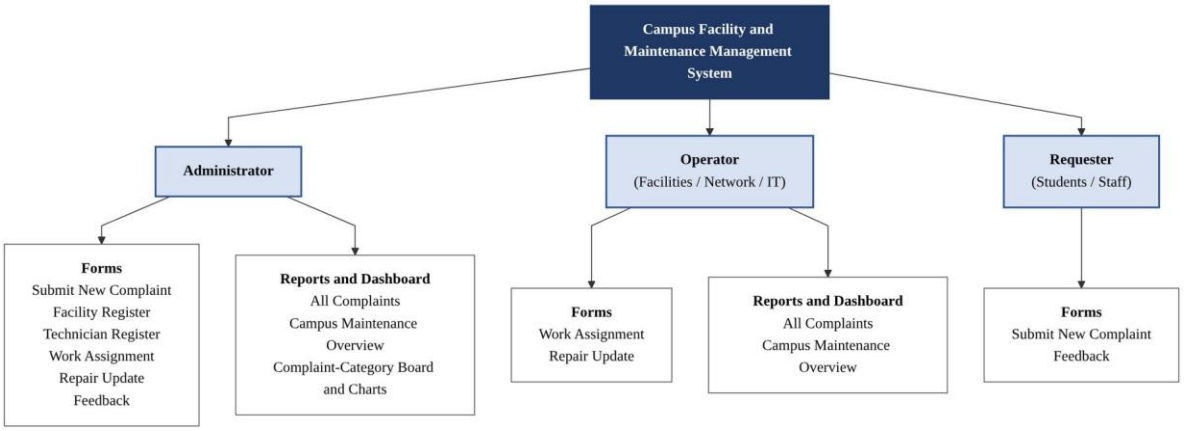
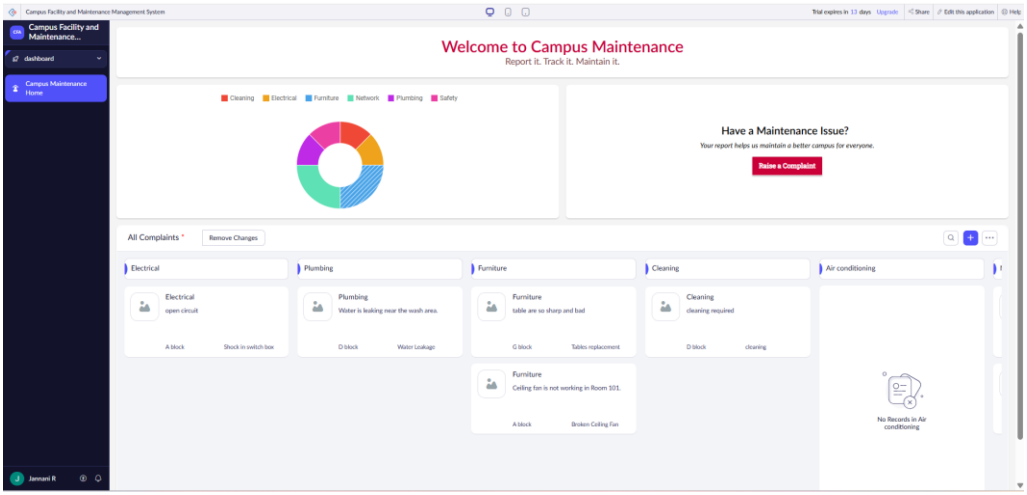


Figure 4.7: Navigation Structure of the Application

The application heading leads to three role-based views. The Administrator can reach all forms, reports, and the dashboard; the Operator works with the Work Assignment and Repair Update forms and the reports; and the Requester uses the Submit New Complaint and Feedback forms. The menus show the intended design; the permissions actually configured are discussed in Section 4.6.

Figure 4.8: Campus Maintenance Overview Dashboard



The dashboard shows the application menu, complaint counts by category, a bar chart of complaints by issue category, a pie chart of complaints by request status, and an All Complaints button that opens the report.

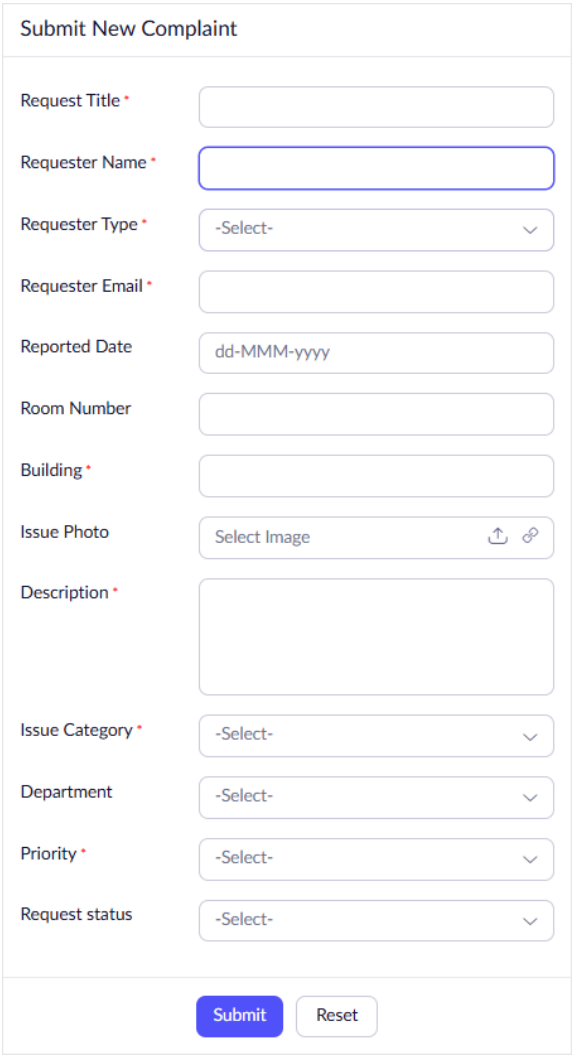


Figure 4.9: Submit New Complaint Form

The form collects the complaint details listed in Table 4.2. Required fields are marked with an asterisk, and the form provides Submit and Reset buttons.

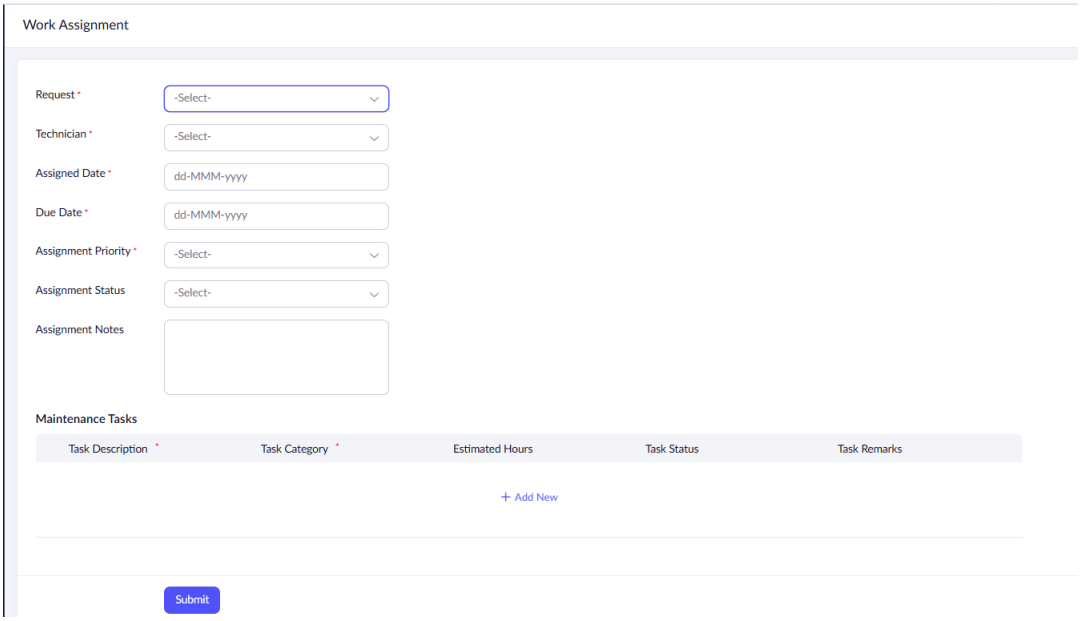


Figure 4.10: Work Assignment Form

The form selects the request and technician, sets the assigned date, due date, priority, and status, and includes the Maintenance Tasks subform for task details. Required fields are marked with an asterisk.

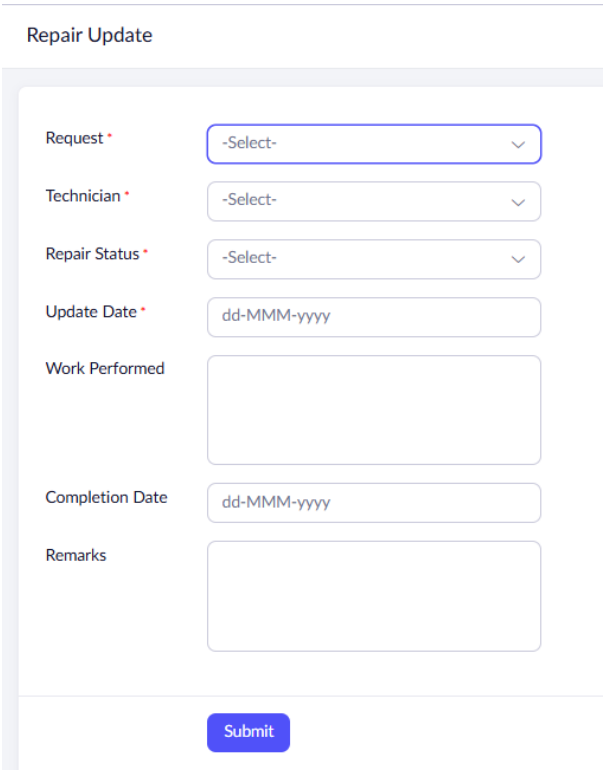


Figure 4.11: Repair Update Form

The form records the repair status, update date, work performed, completion date, and remarks for a selected request and technician.

4.6 Access Control Design

Role-Based Access Control (RBAC) is used to limit each user to the components needed for their role. Table 4.8 gives the intended permissions of the three roles: Administrator, Operator, and Requester / Portal User.

Table 4.8: Role–Permission Matrix

Component	Administrator	Operator	Requester / Portal User
Submit New Complaint	A E V D	—	A
Facility Register	A E V D	—	—
Technician Register	A E V D	—	—
Work Assignment	A E V D	V (To be verified)	—
Repair Update	A E V D	A E V (To be verified)	—
Feedback	A E V D	—	A
All Complaints Report	V	V (To be verified)	—
Campus Maintenance Overview Dashboard	V	V (To be verified)	—

Legend: A = Add; E = Edit; V = View; D = Delete; — = No access; (To be verified) = intended permission not yet confirmed in the configured application.

The matrix records the intended access design. The Administrator is intended to have full management access, the Operator access to the relevant maintenance assignments and repair updates, and the Requester access to complaint submission and feedback. The Operator role previously had an issue viewing Work Assignment records created by the Administrator, so Operator permissions are marked for verification. Record-level restrictions, such as a Requester seeing only their own records, are not claimed as implemented.

RBAC supports security by giving each role only the access its duties require. Administrators manage the master data and assignments, operators handle their maintenance work, and requesters are limited to submitting complaints and feedback. This reduces the risk of unauthorized changes to maintenance records.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The Campus Facility & Maintenance Management System was created using Zoho Creator, a low-code application development platform. The application workspace contains the forms, reports, dashboards, and workflows required to manage campus maintenance activities. The navigation menu provides access to complaint submission, facility registration, technician registration, work assignment, repair updates, and feedback. The application was organized with a simple and user-friendly interface to help administrators and operators access the required modules efficiently. Zoho Creator's built-in configuration features were used to organize the application components and support centralized maintenance management.

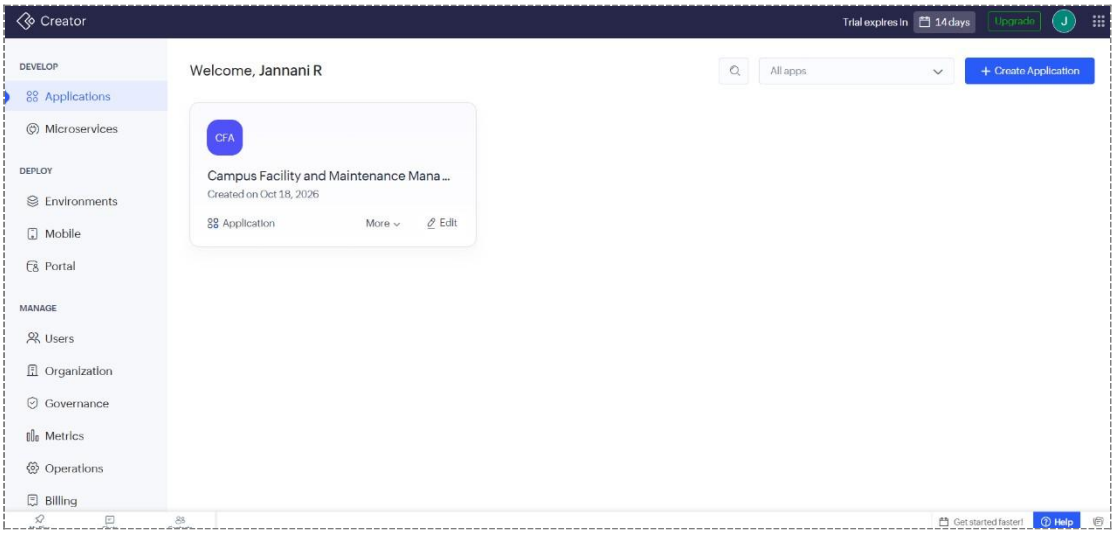


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

The forms in the Campus Facility & Maintenance Management System were developed using Zoho Creator to collect and manage complaint details, facility information, operator details, work assignments, repair updates, and feedback. Lookup fields and subforms are used where required to connect related records and organize maintenance tasks. Deluge validation scripts ensure that the Due Date is not earlier than the Assigned Date and that the Completion Date is provided when a repair is marked as Resolved. The complaint form displays a confirmation message and complaint ID after successful submission.

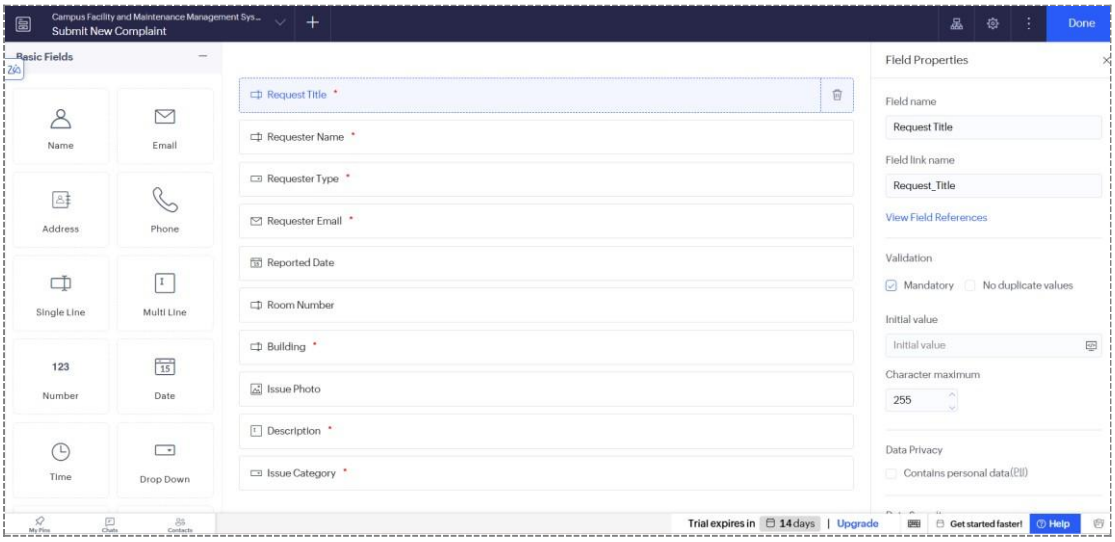


Figure 5.2: [Submit New Complaint] Form in the Form Builder

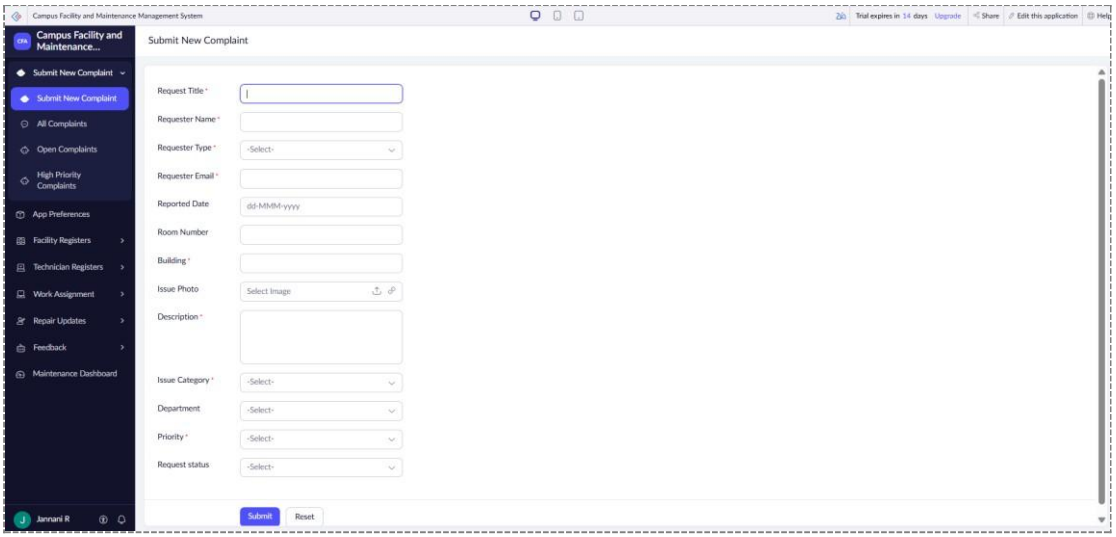


Figure 5.3: Submit New Complaint Form in Live Mode

5.3 Reports

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Complaints	List	Submit New Complaint	Displays complaint records	None, unless configured	Administrator / authorized users
Complaints by Issue Category	Board / Kanban	Submit New Complaint	Grouped by Issue Category	None, unless configured	Authorized users

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Campus Maintenance Overview	Dashboard	Comlaint reports and charts	Visual summaries of complaint information	None, unless configured	Authorized users

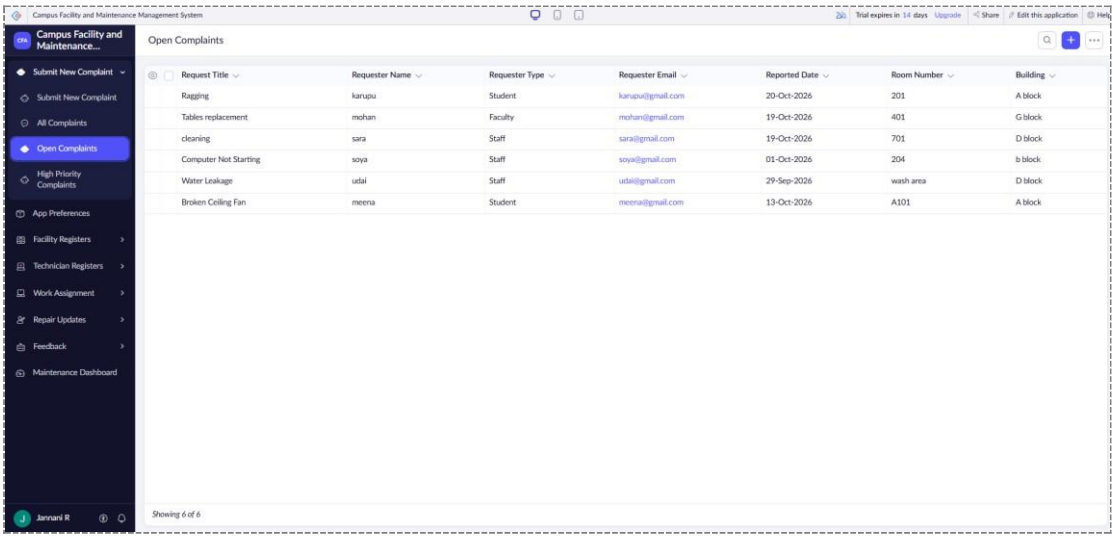


Figure 5.4: All complaint Report

5.4 Pages and Dashboards

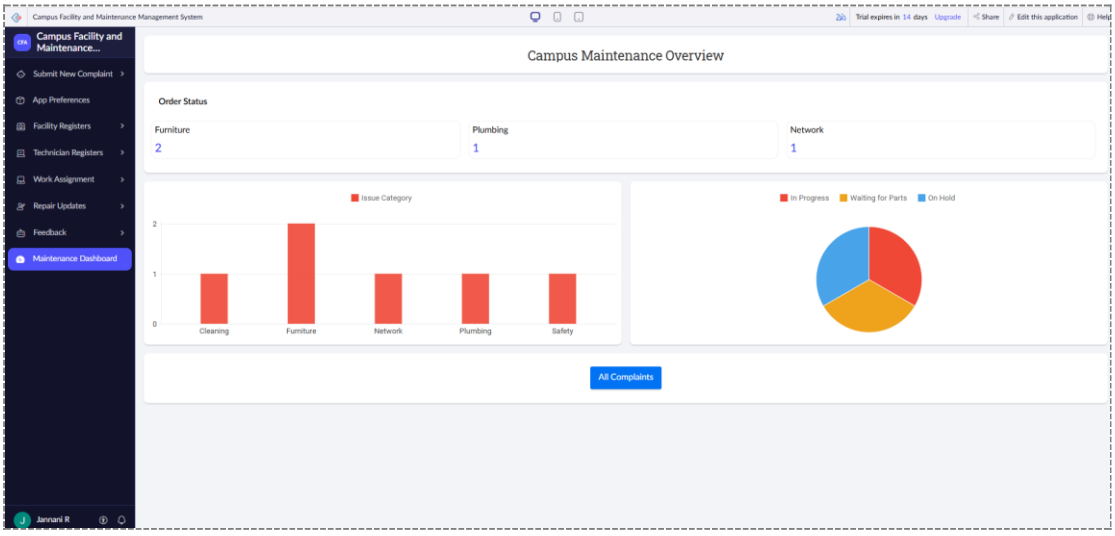


Figure 5.5: Maintanance Dashboard

The Campus Maintenance Overview dashboard was created in Zoho Creator to provide a visual summary of campus maintenance activities. It includes charts and a complaint board grouped by Issue Category, helping administrators monitor reported issues and understand the types of maintenance requests received. The dashboard uses report-based visualizations to present complaint information in an organized format. It is intended for administrators and

other authorized users to support complaint monitoring and maintenance management from a centralized interface.

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Work Assignment – Form validation	Prevent Due Date from being earlier than Assigned Date	if, date comparison, alert, cancel submit
DS-02	Repair Update – Form validation	Require Completion Date when Repair Status is Resolved	if, null checking, alert, cancel submit
DS-03	Submit New Complaint – Successful submission workflow	Display confirmation message and Complaint ID	info, string concatenation, input.ID
DS-04	Work Assignment – Assignment notification workflow	Send an email to the assigned operator	Workflow trigger, email notification

Code 5.1: Work Assignment Date Validation

```
/* DS-01: Work Assignment form validation
   Purpose: Ensure Due Date is not before Assigned Date. */

if(input.Assigned_Date != null && input.Due_Date != null)
{
    if(input.Due_Date < input.Assigned_Date)
    {
        alert "Due Date cannot be earlier than Assigned Date.";
        cancel submit;
    }
}
```

Code 5.2: Repair Completion Validation

```
/* DS-02: Repair Update form validation
   Purpose: Require Completion Date for resolved repairs. */

if(input.Repair_Status == "Resolved" && input.Completion_Date == null)
{
    alert "Please enter the Completion Date before marking the repair
as Resolved.";
    cancel submit;
}
```

Code 5.3: Complaint Submission Confirmation

```
/* DS-03: Complaint submission confirmation
   Purpose: Display a success message and complaint ID. */

info "Campus Maintenance System";
info "A new complaint has been submitted successfully.";
info "Complaint ID: " + input.ID;
```

The Deluge scripts validate maintenance records and improve workflow automation. The Work Assignment validation script prevents submission when the Due Date is earlier than the Assigned Date. The Repair Update validation script ensures that a Completion Date is entered before a repair can be marked as Resolved. The complaint submission script displays a success message along with the generated Complaint ID after successful submission. The operator assignment workflow sends an email notification to the assigned operator. These scripts were tested to verify the validation messages, successful complaint confirmation, and automated notification behavior.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Work Assignment Date Validation	Form Workflow	On Validate	Due Date is earlier than Assigned Date	Display error and cancel submission	DS-01
Repair Completion Validation	Form Workflow	On Validate	Status is Resolved and Completion Date is empty	Display error and cancel submission	DS-02

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Complaint Submission Confirmation	Form Workflow	On Success	Complaint submitted successfully	Display confirmation message and Complaint ID	DS-03

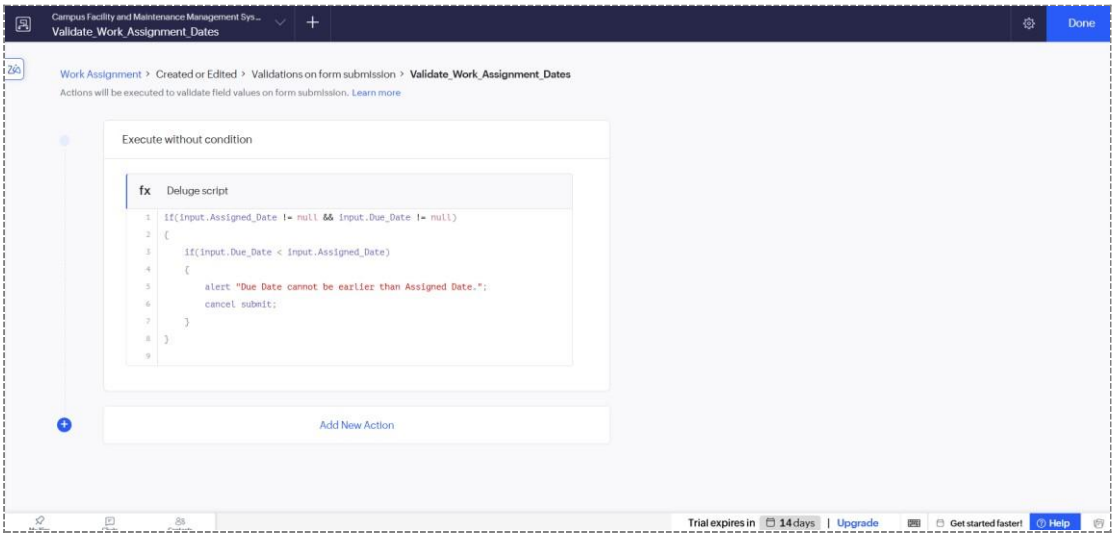


Figure 5.6: Validate_work_Assignment_Dates Configuration

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Complaint Data Sync	Zoho Sheet	Connection + zoho.sheet.createRecords	Configured Zoho connection	Sends complaint details from Zoho Creator to a spreadsheet

Code 5.3: Integration script

```
rowData = Map();
rowData.put("Request Title",input.Request_Title);
rowData.put("Requester Name",input.Requester_Name);
rowData.put("Requester Type",input.Requester_Type);
rowData.put("Requester Email",input.Requester_Email);
rowData.put("Reported Date",input.Reported_Date);
rowData.put("Room Number",input.Room_Number);
rowData.put("Building",input.Building);
rowData.put("Issue Photo","");
rowData.put("Description",input.Description);
rowData.put("Issue Category",input.Issue_Category);
rowData.put("Department",input.Department);
rowData.put("Priority",input.Priority);
rowData.put("Request Status",input.Request_status);
queryData = Map();
response = zoho.sheet.createRecords("t64m8072c37d5aa0247eca4171544f51c04f8","Sheet1",rowData,
queryData,"zoho_sheet_connection");
info response;
```

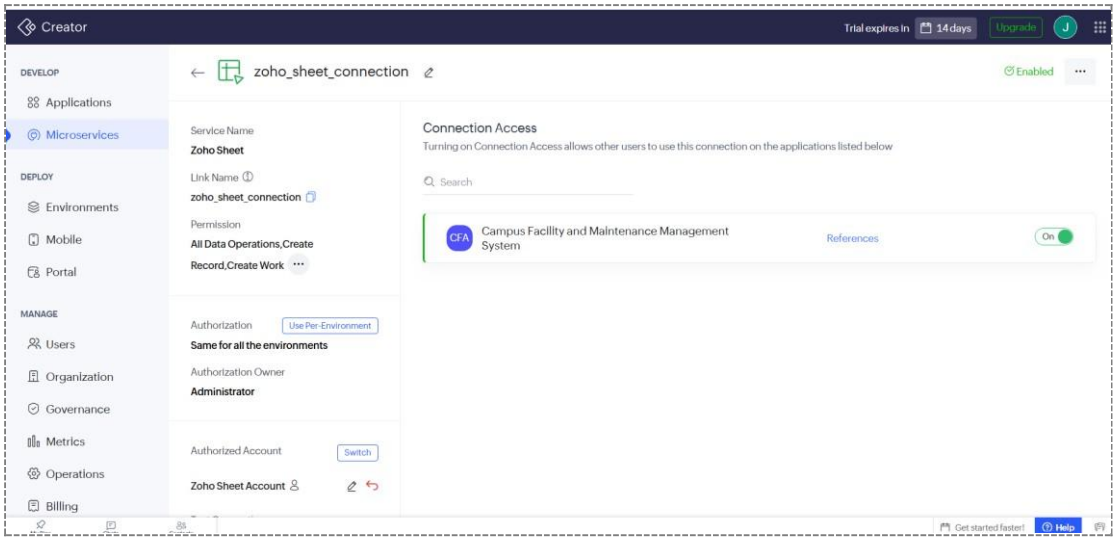


Figure 5.7: Zoho_sheet_connection

5.8 User Management and Role-Based Access

User management in the Campus Facility & Maintenance Management System is designed to support access to maintenance functions according to user responsibilities. The Administrator manages complaint records, facility information, operator details, and work assignments. Operators are responsible for viewing and updating maintenance work as permitted by the application's configuration. Students and staff can submit maintenance complaints through the published complaint form. Zoho Creator's user management and permission settings are used to control access to application components. The published complaint form provides an entry point for users to report campus issues without requiring access to the administrative interface.

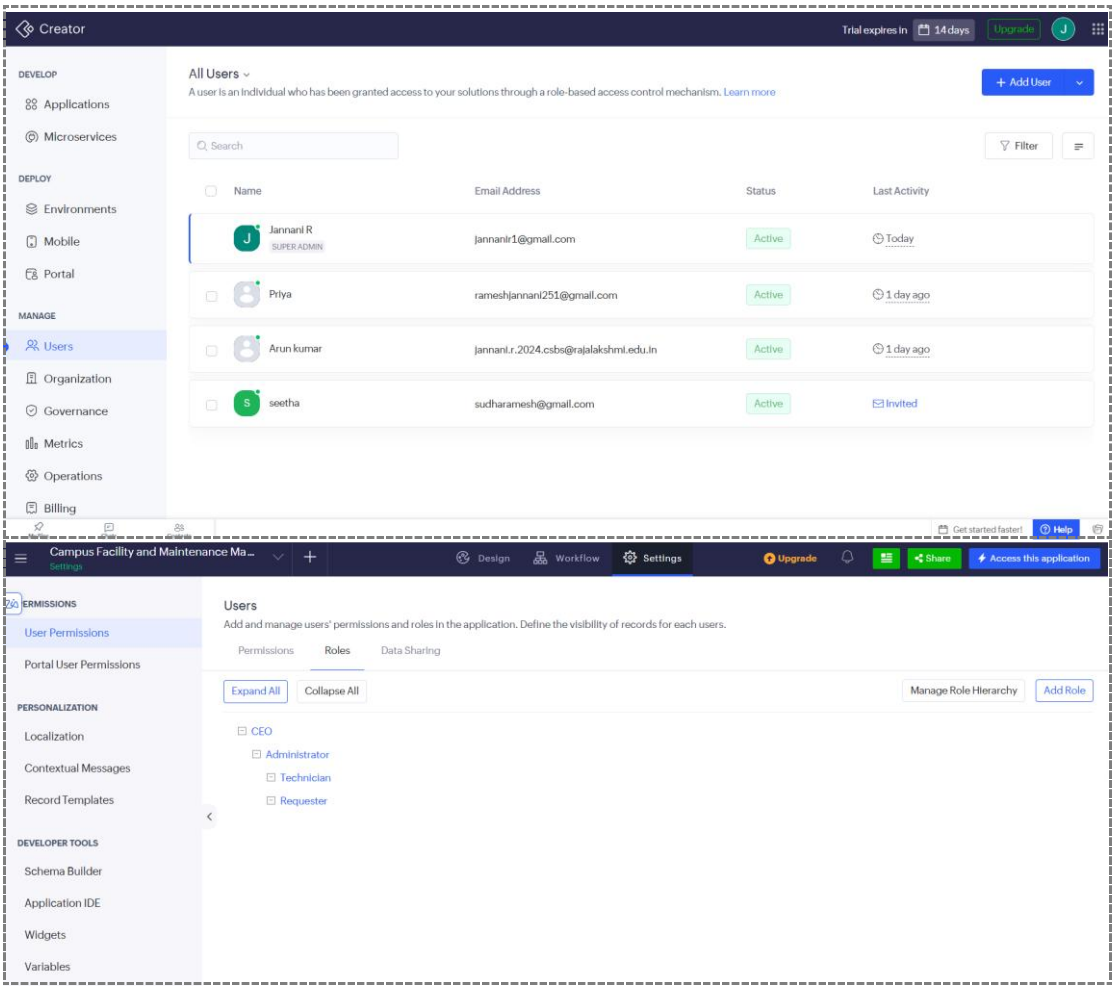


Figure 5.8: User profiles(RBAC) & roles

5.9 AI and Analytics Features

Zoho Creator's Zia AI assistant was used to assist in generating an Entity-Relationship (ER) diagram for the application. The diagram helps represent the main entities, their attributes, and the relationships between maintenance records. The generated diagram should be checked against the actual forms and lookup fields configured in the application.

These features support better data organization and visualization. Advanced AI functions such as complaint prediction, sentiment analysis, and image-based issue detection were not implemented in the current version.

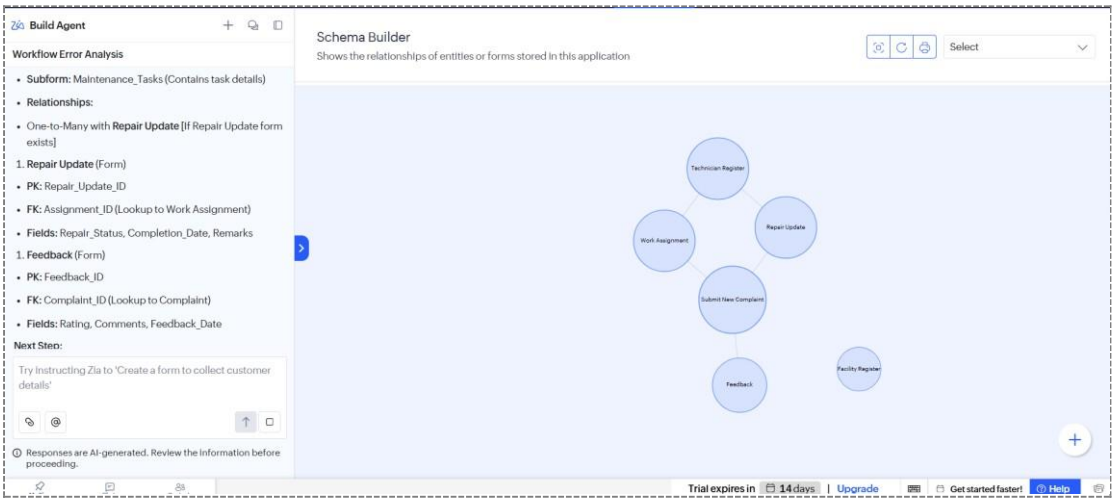


Figure 5.9: AI / Analytics Feature in Use

5.10 Pro-Code Extension

The Campus Facility & Maintenance Management System was developed mainly using Zoho Creator's low-code features.

- Forms, reports, dashboards, and workflows were used to manage maintenance activities.
- Deluge scripting was implemented to validate inputs and automate basic tasks.
- No separate pro-code extensions, such as Zoho Catalyst, custom JavaScript widgets, or Creator REST API integrations, were implemented in the current version.
- The built-in features of Zoho Creator were sufficient to meet the main project requirements.
- In the future, Zoho Catalyst and Creator REST APIs could be used to add advanced features and integrate external applications.

5.11 Mobile and Portal Access

The application was developed and tested using the Zoho Creator web interface. Mobile app access and customer portal configuration were not implemented in the current version. These features may be explored in the future to improve accessibility for students, staff, and maintenance operators

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was carried out to verify the functionality, accuracy, and reliability of the Campus Facility & Maintenance Management System developed using Zoho Creator. The testing process focused on individual forms, Deluge validation scripts, workflows, reports, dashboards, and the Zoho Sheet integration. Different input conditions were considered to identify errors and ensure that the application behaved as expected.

Unit testing was used to check individual forms and validation scripts. Integration testing focused on the complaint submission workflow and the transfer of complaint details to Zoho Sheet. Negative testing was performed by checking invalid date combinations and attempting to mark a repair as resolved without entering the completion date. The application interface and reports were also reviewed to verify that maintenance records were displayed in an organized manner. Further role-based testing and user acceptance testing can be conducted to evaluate access permissions and usability across different user roles.

6.2 Test Cases

Table 6.1: Test Cases

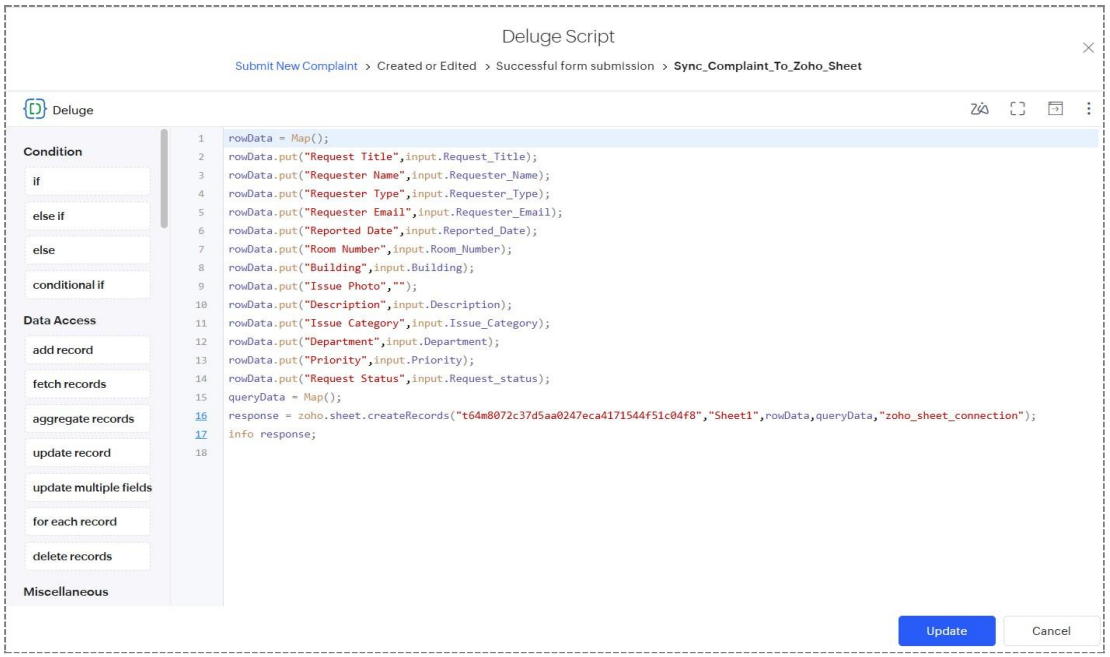
TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Submit a valid complaint	Enter all mandatory fields and submit	Complaint is saved successfully	Complaint record was submitted successfully.	Pass
TC-02	FR-01	Submit a complaint with a mandatory field missing	Leave a required field empty	Required-field validation appears	Mandatory-field validation was displayed.	Pass
TC-03	FR-01	Verify complaint confirmation	Submit a valid complaint	Success message and complaint ID are displayed	Success message and Complaint ID were displayed.	Pass

TC-04	FR-02	Register a facility	Enter facility details and submit	Facility record is saved	Facility record was saved successfully.	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-05	FR-03	Register a technician/operator	Enter required details and submit	Operator record is saved	Technician/operator record was saved successfully.	Pass
TC-06	FR-04	Create a work assignment	Select the relevant complaint and operator and enter dates	Work assignment is saved	Work assignment was created successfully.	Pass

6.3 Debugging Deluge Scripts

Debugging was performed during the development of the Campus Facility & Maintenance Management System to identify and resolve issues in Deluge scripts and workflow configurations. The Deluge editor, info statements, execution responses, and workflow behaviour were used to inspect the execution of scripts. Error messages and the results displayed after form submission were also checked to identify problems and verify the expected behaviour.


Deluge Script
Submit New Complaint > Created or Edited > Successful form submission > Sync_Complaint_To_Zoho_Sheet
Deluge
Condition
if
else if
else
conditional if
Data Access
add record
fetch records
aggregate records
update record
update multiple fields
for each record
delete records
Miscellaneous
rowData = Map();
rowData.put("Request Title",input.Request_Title);
rowData.put("Requester Name",input.Requester_Name);
rowData.put("Requester Type",input.Requester_Type);
rowData.put("Requester Email",input.Requester_Email);
rowData.put("Reported Date",input.Reported_Date);
rowData.put("Room Number",input.Room_Number);
rowData.put("Building",input.Building);
rowData.put("Issue Photo","");
rowData.put("Description",input.Description);
rowData.put("Issue Category",input.Issue_Category);
rowData.put("Department",input.Department);
rowData.put("Priority",input.Priority);
rowData.put("Request Status",input.Request_status);
queryData = Map();
response = zoho.sheet.createRecords("t64m8072c37d5aa0247eca4171544f51c04f8","Sheet1",rowData,queryData,"zoho_sheet_connection");
info response;
Update
Cancel

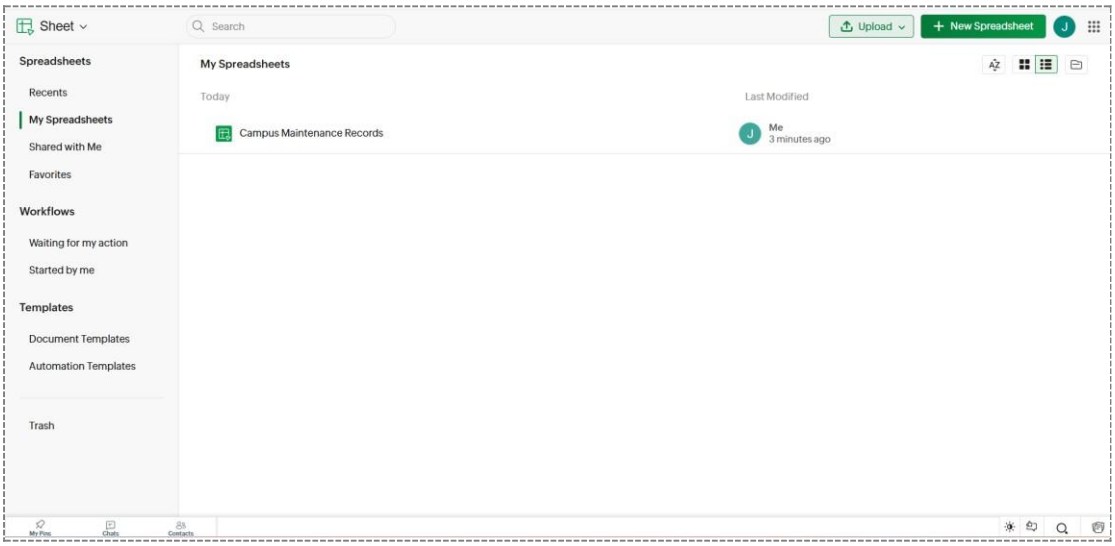


Figure 6.1: Deluge Debug

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Complaint data did not fully sync to Zoho Sheet.	Field mapping issue.	Reviewed integration code and field mapping.	Partially resolved

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
User 1	Student / Complaint Submitter	The complaint form was simple to understand and use.	Reviewed the form layout and required fields.
User 2	Administrator	The dashboard and reports helped organize complaint records.	Reviewed the report and dashboard presentation.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The Campus Facility and Maintenance Management System was configured using Zoho Creator's Development, Stage, and Production environments. These environments support application development, testing, and live deployment. The application was accessed successfully through all three environments, and the version history showed a published package named Default Package 1.0, identified as application version V1.0. The package was classified as a major release and recorded on 19 October 2026.

Version history provides a reference for tracking published application versions. Before future updates are deployed, changes should be reviewed and tested in the appropriate environment to reduce the risk of errors in the live application.

Maintaining version records and preparing a rollback plan can further support recovery if a future release causes unexpected issues.

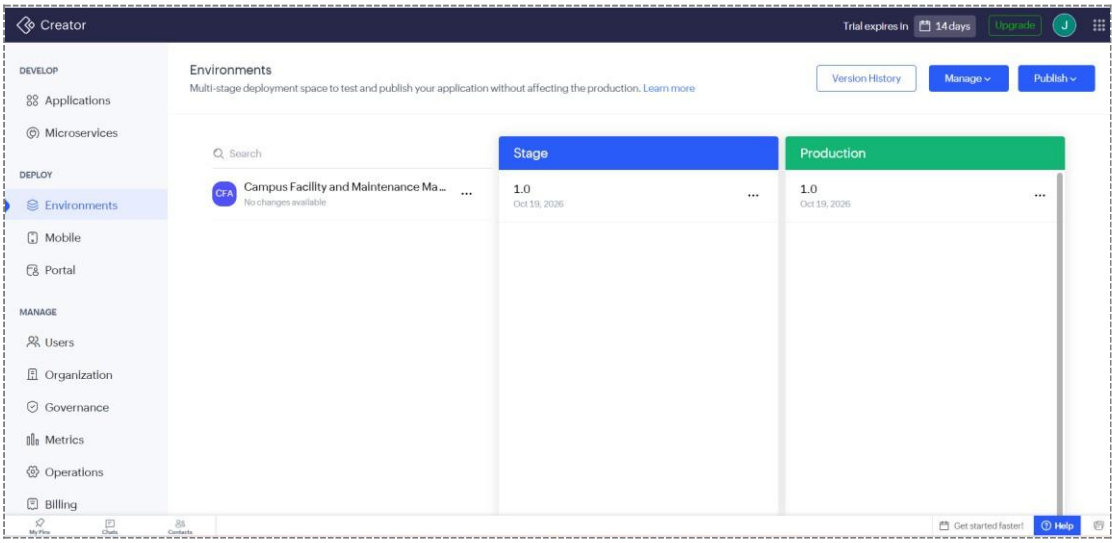


Figure 7.1: Environment

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	Application accessed and checked in Development and Stage	✓
2	Test data reviewed and separated from actual records	✓

3	User roles and permissions configured and verified in Production	✓
4	Zoho Sheet connection authorised and integration checked	✓
5	Configured workflows and validation scripts checked	✓

S.No.	Checklist Item	Done (✓)
6	Complaint form published and URL verified	✓
7	Application version V1.0 recorded in Version History	✓

7.2 Performance Optimisation.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Organized related maintenance tasks using a subform	Work Assignment	Maintenance tasks managed separately	Related tasks grouped within the assignment	Better organization of maintenance data
Displayed complaint records in a structured report	All Complaints Report	Complaint details stored as individual records	Records displayed in a centralized report	Easier record monitoring
Presented complaint information using charts and categories	Campus Maintenance Overview	Records viewed individually	Visual summaries and category-based board	Easier identification of complaint trends
Integrated complaint records with Zoho Sheet	Zoho Sheet Integration	Complaint records maintained in Zoho Creator	Configured complaint details also transferred to a spreadsheet	Additional record tracking

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role/profile permissions	Zoho Creator application and administrative modules; verify configuration	Unauthorized application access
Record-level permission criteria	Complaint, work assignment, and repair records; configure and test if required	Exposure of sensitive data
OAuth connection	Zoho Sheet integration using the configured connection	Credential leakage from hard-coded keys
Encrypted fields	Sensitive data fields; configure if applicable	Exposure of sensitive information

7.4 Governance and Administration

Governance and administration practices are important for maintaining the reliability, consistency, and security of the Campus Facility and Maintenance Management System. Zoho Creator provides application management features such as version history and configuration settings to support maintenance and change tracking. The published application package, V1.0, is recorded in the version history, providing a reference for identifying the released version.

Consistent naming conventions are followed for forms, reports, workflows, and application components to make the system easier to understand and maintain. Application changes should be reviewed and tested before publishing to reduce the possibility of errors. Regular data exports or backups should be maintained to support recovery when necessary. User accounts and licences should also be reviewed periodically to ensure that access is provided only to users who require it.

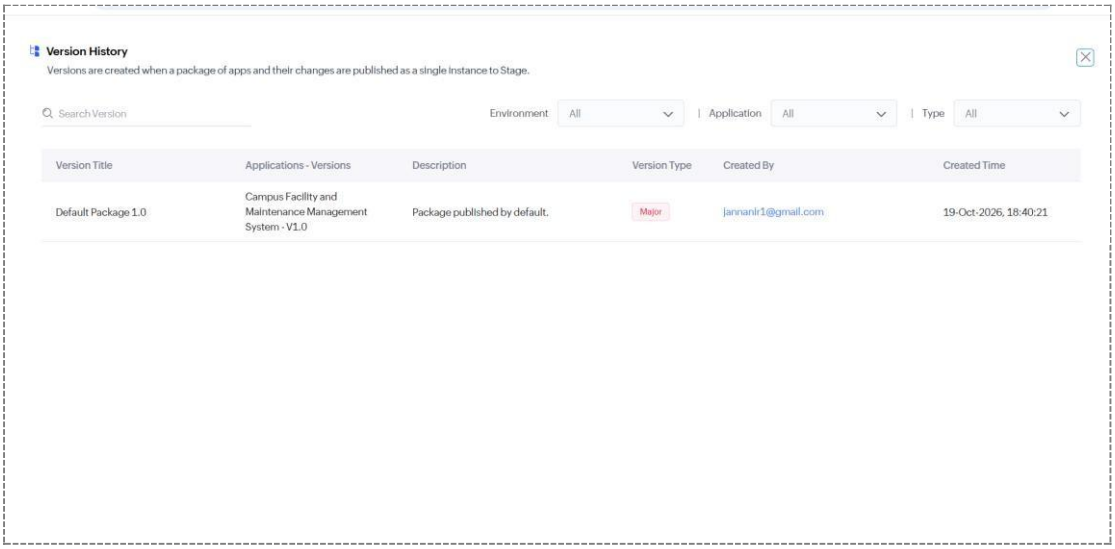


Figure 7.2: Version History and Application Administration in Zoho

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

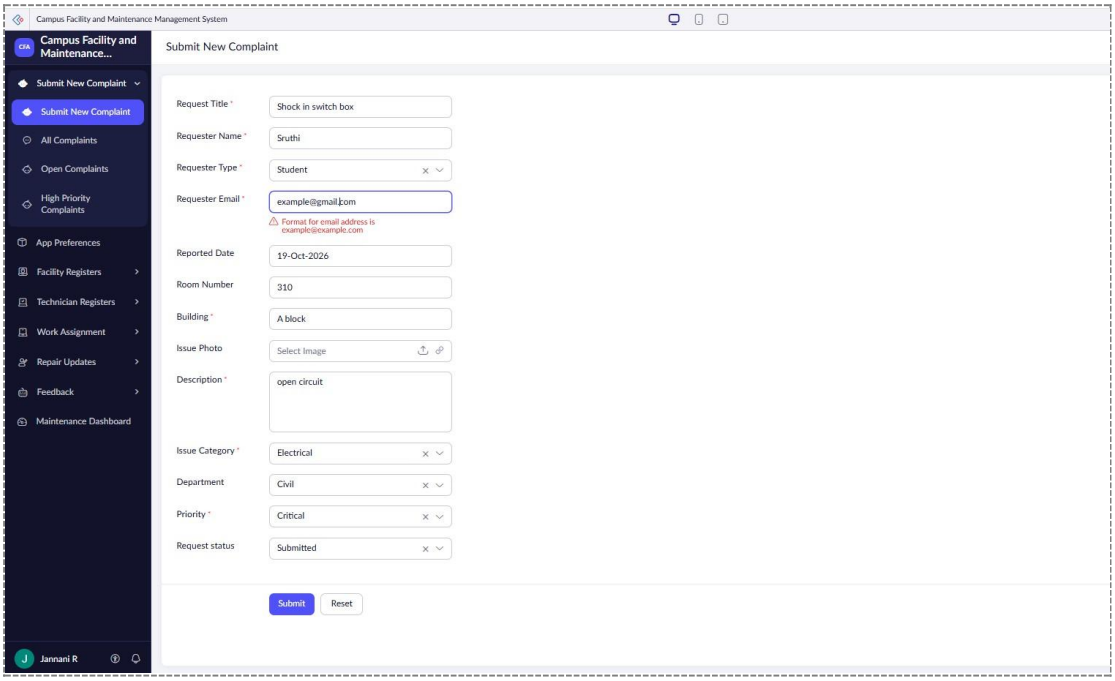


Figure 8.1: Scenario Step 1 – Complaint Submission and Confirmation

A student or staff member opens the published complaint form and enters the required details, such as the issue category, facility information, and complaint description. After successful submission, the application displays a confirmation message along with the generated Complaint ID

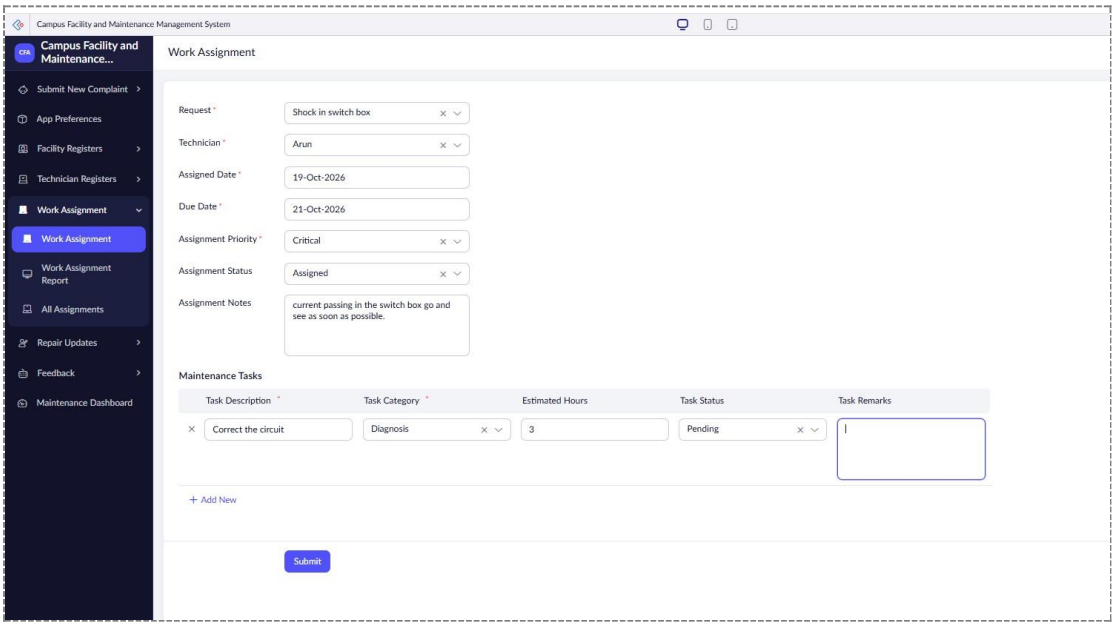
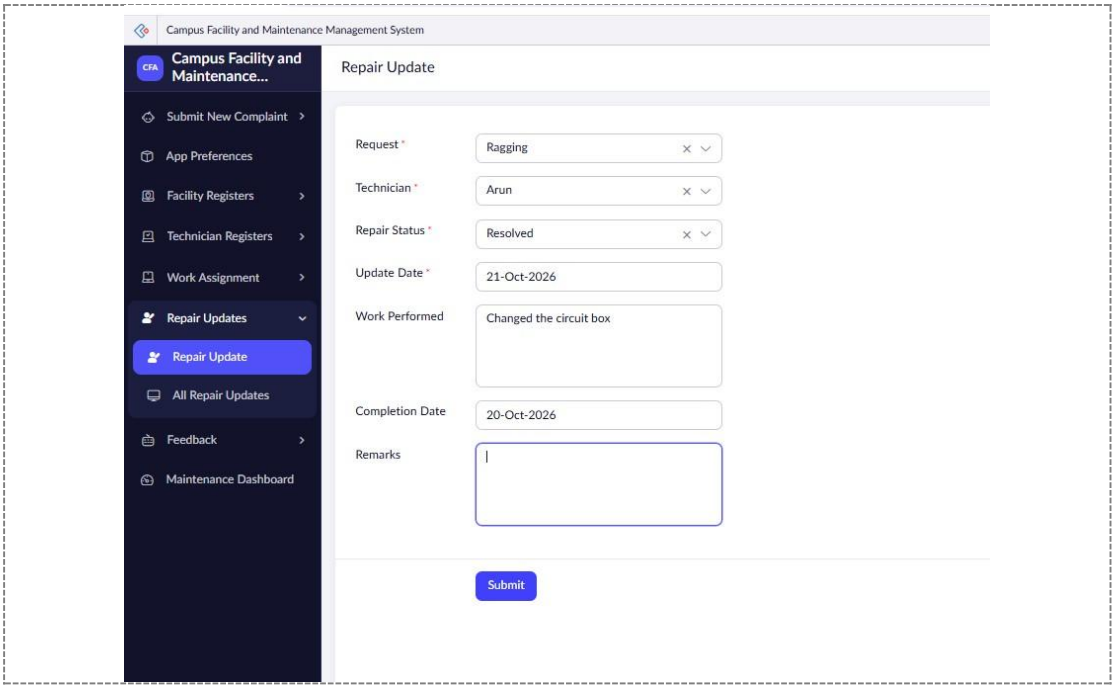


Figure 8.2: Scenario Step 2 –Complaint Review and Work Assignment

The administrator opens the All Complaints report to review the submitted complaint. The maintenance work can then be recorded through the Work Assignment form by selecting the relevant complaint, assigning an operator, and entering the assigned and due dates. The application validates the dates before accepting the assignment.



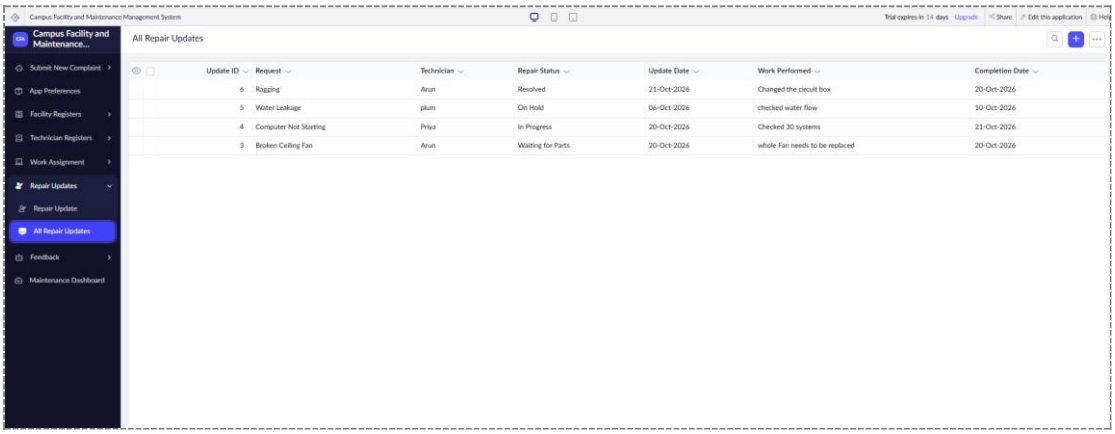


Figure 8.3: Scenario Step 3 – Repair Update and Dashboard Monitoring

Result: The walkthrough demonstrates the main workflow of the application, from reporting a campus maintenance issue to assigning work, updating repair details, and monitoring complaint records. It shows how Zoho Creator forms, validation scripts, reports, and dashboards work together to organize maintenance activities.

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Objective 1: To provide a centralized system for submitting and managing campus maintenance complaints.	Complaint submission form, Facility Register, All Complaints report (Section 5.2 and 5.3).	Complaint details can be collected and viewed in a centralized report.
Objective 2: To organize maintenance work assignments and track repair progress.	Work Assignment form, Repair Update form, and Deluge validation workflows (Sections 5.5 and 5.6).	Assignment dates are validated, and a Completion Date is required when a repair is marked as Resolved.
Objective 3: To improve maintenance monitoring through reports, dashboards, and data integration.	Campus Maintenance Overview dashboard, board grouped by Issue Category, and Zoho Sheet integration (Sections 5.3, 5.4, and 5.7).	Complaint information is presented visually, and configured complaint details are transferred to Zoho Sheet for additional tracking.

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to process one request	Complaints may require manual recording and follow-up.	Complaints can be submitted through a published online form.	Simplifies complaint submission and record creation.
Manual steps	Complaint details and updates may be recorded across paper forms or separate files.	Forms and validation workflows organize data entry and basic checks.	Reduces some repetitive manual recording and checking.
Report generation	Information may need to be collected and summarized manually.	Zoho Creator reports and dashboards display complaint records and visual summaries.	Makes complaint monitoring more organized.
Data accessibility	Records may be distributed across different files or communication channels.	Complaint records are available through the configured application reports, with additional tracking through Zoho Sheet.	Provides a more centralized way to view and track records.

8.4 Limitations of the Low-Code Approach

The Campus Facility and Maintenance Management System was developed using Zoho Creator, which simplifies application development through built-in forms, reports, dashboards, and workflows. However, the low-code approach also has certain limitations in customization, scripting, integration, and platform dependency.

One limitation is UI customization. Although Zoho Creator provides ready-made components and layout options, achieving complete control over the interface may require additional customization. This was addressed by using the available form

layouts, reports, and dashboard components to maintain a simple and organized interface.

Another limitation is scripting complexity. Deluge supports form validation and workflow automation, but complex business logic may require additional scripting and careful testing. This project used Deluge scripts for validating work assignment dates, checking completion dates, and displaying complaint confirmation messages.

Integration with external applications also requires correct connection authorization, field mapping, and configuration. During development, complaint details were transferred to Zoho Sheet, but the Issue Photo field did not synchronize as expected. This highlighted the need to verify integration responses and field mappings rather than assuming that every field transfers successfully.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The development of the Campus Facility and Maintenance Management System provided practical experience with the no-code, low-code, and pro-code approaches to application development. Each approach serves a different purpose depending on the complexity and customization requirements of the application.

No-Code: Zoho Creator's built-in features were used to create forms, configure fields and lookup relationships, design reports, and build dashboards. These tasks required mainly visual configuration rather than traditional programming. This approach helped speed up the development process and organize maintenance information efficiently.

Low-Code: Deluge scripting was used where built-in configuration alone was insufficient. Scripts were implemented to validate work assignment dates, require a Completion Date before marking a repair as Resolved, and display a confirmation message with the generated Complaint ID. The Zoho Sheet integration also connected the application with an external spreadsheet through a configured connection and integration task.

Pro-Code: The current version did not require a separate pro-code extension. However, advanced requirements such as a highly customized user interface, complex predictive maintenance algorithms, or integrations with external systems beyond the platform's

built-in capabilities might require technologies such as JavaScript, Python, or external APIs, depending on the specific requirement.

This project demonstrated that no-code is suitable for standard application components, low-code is useful for custom validation and automation, and pro-code is appropriate when requirements exceed the platform's built-in capabilities. In the future, I would consider moving selected components to pro-code only when there is a clear need for greater flexibility, advanced processing, or integration capabilities that cannot be achieved efficiently with Zoho Creator. This approach helps balance development speed, maintainability, and technical flexibility.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Campus Facility & Maintenance Management System was developed using Zoho Creator to simplify and organize campus maintenance activities. The application includes forms for complaint submission, facility registration, technician registration, work assignment, repair updates, and feedback. Reports, dashboards, and a board grouped by Issue Category help organize and monitor complaints. Deluge scripts were used to validate dates, ensure completion details are entered, and display complaint confirmation messages with generated complaint IDs.

The integration with Zoho Sheet provides an additional way to maintain complaint records. Zoho Creator's low-code features helped develop the application with minimal traditional programming. Overall, the project demonstrates how low-code application development can support efficient complaint management, data organization, and maintenance tracking within a campus environment.

9.2 Future Enhancements

Enhancement 1 – Mobile Application Access: Improve accessibility by enabling maintenance staff and administrators to manage complaints and update repair details through mobile devices.

Enhancement 2 – Automated Notifications: Introduce email or other notifications to inform operators about new work assignments, approaching due dates, and completed repairs.

Enhancement 3 – Advanced Reports and Analytics: Add charts and performance reports to analyze complaint resolution time, recurring maintenance issues, and overall maintenance trends.

Enhancement 4 – Enhanced Role-Based Access: Configure and test user permissions to ensure that administrators, operators, and other users can access the appropriate records and application functions.

9.3 Learning Outcomes and Career Relevance

This project provided practical experience in low-code application development using Zoho Creator. I learned how to create forms, configure lookup fields, organize reports

and dashboards, use Deluge scripting for validation, and integrate Zoho Creator with Zoho Sheet. I also gained experience in identifying application requirements, organizing data, testing workflows, and troubleshooting implementation issues.

The project helped me understand how low-code platforms reduce development time while allowing users to build useful business applications. These skills are relevant to career pathways such as Low-Code Developer, Citizen Developer, Solutions Consultant, and Automation Engineer. In the future, I can build on this experience by learning advanced workflow automation, API integration, and AI-assisted application development.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 13-Oct-2026).

[3] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 13-Oct-2026).

[4] Zoho Corporation, "Zoho Creator Learning Centre." [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 16-Oct-2026)

[5] Gartner, Inc., "Magic Quadrant for Enterprise Low-Code Application Platforms," *Gartner Research*, 2026.

[6] M. Bucaioni, A. Cicchetti, and F. Ciccozzi, "Modelling in low-code development: A multi-vocal systematic review," *IEEE Transactions on Software Engineering*, vol. 21, pp. 1959–1981, 2022.

[7] M. Sanchis, V. García-Perales, F. Fraile, and R. Poler, "Low-code as enabler of digital transformation in manufacturing industry," Applied Sciences, vol. 10, no. 1, Art. no. 12, 2020.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Application navigation	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	Complaint and maintenance forms	5.2	Fig. 5.2–5.3
II	Lookup field (relationship between forms)	Y	Work Assignment and related forms	4.2, 5.2	Fig. 5.2
II	Subform (parent–child data)	Y	Work Assignment – maintenance tasks	4.2, 5.2	Fig. 5.2
II	Field validations / layout rules	Y	Work Assignment and Repair Update	5.2	Code DS-01, DS-02
II	List report with filter, grouping, custom action	Y	All Complaints report	5.3	Fig. 5.4
II	Kanban / Calendar / Timeline / Map report	N	Complaints board grouped by Issue Category	5.3	-
II	Pivot table / chart report	Y	Campus Maintenance Overview dashboard	5.3	Fig. 5.5
II	Page / dashboard built with UI Builder	Y	Campus Maintenance Overview	5.4	Fig. 5.5
II	Deluge – form events (validate / success / field actions)	Y	Work Assignment, Repair Update, Submit New Complaint	5.5	DS-01–DS-03
II	Deluge – functions, fetch, loops, collections	N	No confirmed implementation	5.5	-
II	Email / in-app notifications	N	Not implemented	5.5, 5.6	-
II	Form workflow	Y	Validation and complaint confirmation workflows	5.6	Fig. 5.6
II	Scheduled workflow	N	Not implemented	5.6	-
II	Approval process	N	Not implemented	5.6	-

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Blueprint	N	Not implemented	5.6	-
II	Batch / report workflow	N	Not implemented	5.6	-
II	Testing & debugging evidence	Y	Form validations and Zoho Sheet integration troubleshooting	6	Fig. 6.1
III	Integration with a Zoho service	Y	Zoho Sheet integration	5.7	Fig. 5.7
III	Connection / REST API call (invokeurl)	Y	Configured Zoho Sheet connection; no confirmed invokeurl	5.7	Fig. 5.7
III	Webhook or Creator REST API	N	Not implemented	5.7	-
III	Users, roles and permission profiles	Y	Application user-management settings	5.8	Fig. 5.8
III	Record-level access criteria / customer portal	N	Not confirmed	5.8	-
III	Zia / AI feature	Y	ER diagram generation using Zia.	5.9	Fig. 5.9
III	Analytics (charts / Zoho Analytics)	Y	Campus Maintenance Overview dashboard charts	5.9	Fig. 5.5
IV	Performance optimisation applied & measured	N	Functional/organizational improvements; no measured performance results	7.2	Table 7.2
IV	Deployment across environments	Y	Development, Stage, Production and Version History	7.1	Fig.7.1
IV	Security controls (encryption, OAuth, permissions)	N	Zoho Creator connection and security settings	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Version History and administration settings	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	No separate pro-code extension implemented	5.10	-

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
V	Low-code limitations & spectrum analysis	Y	Discussion of limitations and development approaches	8.4, 8.5	Sections 8.4-8.5
—	Mobile app / portal access	N	Not implemented in current version	5.11	-

Total features implemented: 20 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Work Assignment – Date Validation

```
// Script ID : [DS-01]
// Location: Work Assignment form → On Validate
// Purpose   : Prevent the Due Date from being earlier than the
Assigned Date.


if(input.Assigned_Date != null && input.Due_Date != null)
{
    if(input.Due_Date < input.Assigned_Date)
    {
        alert "Due Date cannot be earlier than Assigned Date.";
        cancel submit;
    }
}
```

C.2 Repair Update – Completion Date Validation

```
// Script ID : [DS-02]
// Location  : Repair Update form → On Validate
// Purpose   : Require a Completion Date when the repair status is
Resolved.


if(input.Repair_Status == "Resolved" && input.Completion_Date == null)
{
    alert "Please enter the Completion Date before marking the repair
as Resolved.";
    cancel submit;
}
```

C.3 Submit New Complaint – Confirmation Message

```
// Script ID : [DS-02]
// Location  : Submit New Complaint form → On Success
// Purpose   :Display a successful submission message and the
generated Complaint ID.



info "Campus Maintenance System";
info "A new complaint has been submitted successfully.";
info "Complaint ID: " + input.ID;
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Step 1: Open a web browser on your computer or mobile device.

Step 2: Open the published complaint form using the URL provided by the administrator.

Step 3: Students and staff can access the complaint form directly. Administrators and authorized operators must sign in to Zoho Creator using their registered accounts.

Step 4: Open the Campus Facility and Maintenance Management System from the application list.

Step 5: Use the navigation menu to access the available forms, reports, and dashboard.

D.2 Instructions for Administrator

Step 1: Sign in to Zoho Creator using your authorized account.

Step 2: Open the Campus Facility and Maintenance Management System.

Step 3: Select All Complaints to view submitted maintenance complaints.

Step 4: Open Facility Register or Technician Register to manage facility and operator information.

Step 5: Open Work Assignment, select the relevant complaint and operator, and enter the assigned and due dates.

Step 6: Open Repair Update to update the repair status and enter the completion date when the repair is resolved.

Step 7: Open Campus Maintenance Overview to monitor complaints through charts and the board grouped by Issue Category.

Step 8: Check the connected Zoho Sheet to review synchronized complaint details.

D.3 Instructions for Student/Staff

Step 1: Open the published complaint form using the provided URL.

Step 2: Enter the required facility details and select the appropriate issue category.

Step 3: Enter a clear description of the maintenance problem.

Step 4: Upload an issue photo if required and available.

Step 5: Review the entered information for accuracy.

Step 6: Click Submit to register the complaint.

Step 7: Read the success message and note the generated Complaint ID.

Step 8: Contact the administrator for a complaint status update if direct tracking is unavailable.